BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HỒ CHÍ MINH (HUTECH)
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

MÔN: LẬP TRÌNH MẠNG (NETWORKING PROGRAMMING)

ĐỀ TÀI:

XÂY DỰNG HỆ THỐNG BẦU CỬ ĐIỆN TỬ E-VOTING PHÂN TÁN DỰA TRÊN CÔNG NGHỆ BLOCKCHAIN VỚI GIAO THỨC ĐỒNG THUẬN PROOF-OF-WORK VÀ MẠNG NGANG HÀNG P2P TCP/IP TRÊN NỀN TẢNG JAVA 21

Ngành đào tạo:	CÔNG NGHỆ THÔNG TIN
Chuyên ngành:	KỸ THUẬT PHẦN MỀM & HỆ THỐNG MẠNG
Giảng viên hướng dẫn:	ThS. Huỳnh Đắc Phước
Sinh viên thực hiện 1:	**Nguyễn Hoàng Nhật – MSSV: 2380601549 (Nhóm trưởng)**
Sinh viên thực hiện 2:	Nguyễn Đỗ Trung Lương – MSSV: 2380601234
Sinh viên thực hiện 3:	Lê Quang Minh – MSSV: 2380605678
Lớp học phần:	23DTHB4 – Khóa: 2023 – 2027

TP. HỒ CHÍ MINH, THÁNG 10 NĂM 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HỒ CHÍ MINH (HUTECH)
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

MÔN: LẬP TRÌNH MẠNG (NETWORKING PROGRAMMING)

ĐỀ TÀI:

XÂY DỰNG HỆ THỐNG BẦU CỬ ĐIỆN TỬ E-VOTING PHÂN TÁN DỰA TRÊN CÔNG NGHỆ BLOCKCHAIN VỚI GIAO THỨC ĐỒNG THUẬN PROOF-OF-WORK VÀ MẠNG NGANG HÀNG P2P TCP/IP TRÊN NỀN TẢNG JAVA 21

Ngành đào tạo:	CÔNG NGHỆ THÔNG TIN
Chuyên ngành:	KỸ THUẬT PHẦN MỀM & HỆ THỐNG MẠNG
Giảng viên hướng dẫn:	ThS. Huỳnh Đắc Phước
Sinh viên thực hiện 1:	**Nguyễn Hoàng Nhật – MSSV: 2380601549 (Nhóm trưởng)**
Sinh viên thực hiện 2:	Nguyễn Đỗ Trung Lương – MSSV: 2380601234
Sinh viên thực hiện 3:	Lê Quang Minh – MSSV: 2380605678
Lớp học phần:	23DTHB4 – Khóa: 2023 – 2027

TP. HỒ CHÍ MINH, THÁNG 10 NĂM 2026

PHIẾU ĐÁNH GIÁ KẾT QUẢ ĐỒ ÁN

(Dành cho Giảng viên hướng dẫn chấm điểm học phần Lập trình mạng)

Tên đề tài: Xây dựng hệ thống Bầu cử điện tử E-Voting phân tán dựa trên công nghệ Blockchain với giao thức đồng thuận Proof-of-Work và mạng ngang hàng P2P TCP/IP trên nền tảng Java 21

Giảng viên hướng dẫn: ThS. Huỳnh Đắc Phước – Bộ môn Mạng máy tính & An ninh thông tin

Sinh viên thực hiện: Nguyễn Hoàng Nhật (2380601549 - Nhóm trưởng), Nguyễn Đỗ Trung Lương (2380601234), Lê Quang Minh (2380605678) – Lớp: 23DTHB4

STT	Tiêu chí đánh giá chi tiết	Điểm tối đa	Điểm đạt	Ghi chú
1	Khảo sát, phân tích yêu cầu và cơ sở lý thuyết về mạng TCP/IP & P2P	1.5		Hiểu sâu bản chất Socket
2	Thiết kế kiến trúc hệ thống, giao thức đóng khung gói tin (Framing)	2.0		Xử lý chống dính gói tin
3	Hiện thực hóa tầng mạng P2P Sockets, đa luồng Java 21 Virtual Threads	2.5		Mạng Mesh 10 nodes, Gossip
4	Hiện thực hóa Blockchain, Merkle Tree, Proof-of-Work, Chữ ký số ECDSA	2.0		Sổ cái chuẩn Bitcoin
5	Hiện thực giao diện trực quan hóa, kiểm toán an ninh và giả lập tấn công	1.0		Trực quan sinh động
6	Hình thức trình bày báo cáo đúng chuẩn học thuật, kỹ năng vấn đáp	1.0		Báo cáo công phu, chuẩn mực
	TỔNG CỘNG ĐIỂM ĐÁNH GIÁ TOÀN DIỆN	10.0		

Nhận xét chi tiết của Giảng viên hướng dẫn:

.....
.....

Chủ tịch Hội đồng chấm điểm
(Ký và ghi rõ họ tên)

TP.HCM, Ngày ..... tháng 10 năm 2026
Giảng viên hướng dẫn
(Ký và ghi rõ họ tên)

LỜI CAM ĐOAN

Kính thưa Ban Giám hiệu, Khoa Công nghệ Thông tin Trường Đại học Công nghệ TP. Hồ Chí Minh (HUTECH) cùng quý Thầy/Cô phụ trách bộ môn Lập trình mạng.

Chúng tôi – nhóm sinh viên thực hiện đề tài gồm các thành viên: Nguyễn Hoàng Nhật (MSSV: 2380601549 - Nhóm trưởng), Nguyễn Đỗ Trung Lương (MSSV: 2380601234) và Lê Quang Minh (MSSV: 2380605678), sinh viên lớp 23DTHB4, Khóa 2023 – 2027 – xin cam đoan các nội dung sau:

- **1. Tính trung thực khoa học:** Đề tài nghiên cứu và xây dựng hệ thống: "Xây dựng hệ thống Bầu cử điện tử E-Voting phân tán dựa trên công nghệ Blockchain với giao thức đồng thuận Proof-of-Work và mạng ngang hàng P2P TCP/IP trên nền tảng Java 21" là công trình nghiên cứu khoa học và phát triển phần mềm được thực hiện hoàn toàn độc lập, trung thực bởi nhóm tác giả dưới sự định hướng, hỗ trợ và hướng dẫn của ThS. Huỳnh Đắc Phước.
- **2. Tính nguyên bản của mã nguồn:** Toàn bộ kiến trúc mạng ngang hàng P2P, cơ chế đóng khung luồng tin độ dài cố định Length-Prefixed Stream Framing, cấu trúc giải mã gói tin NetworkMessage, máy chủ đa luồng Java 21 Virtual Threads, sổ cái Blockchain chuẩn Bitcoin, thuật toán đào Proof-of-Work và giao diện giám sát trực quan đều được nhóm tự tay thiết kế, viết mã nguồn (from scratch) bằng ngôn ngữ Java 21 thuần, không sao chép nguyên mẫu hoặc sử dụng trái phép mã nguồn của bất kỳ cá nhân hay tổ chức nào khác.
- **3. Quy chuẩn trích dẫn học thuật:** Mọi tài liệu khoa học, số liệu thống kê, bài báo quốc tế của Satoshi Nakamoto (2008), các chuẩn kỹ thuật RFC của IETF về TCP/IP, các tài liệu JEP của Oracle về Java Virtual Threads và mã nguồn tham khảo đều được trích dẫn nguồn gốc rõ ràng, minh bạch tại phần Tài liệu tham khảo theo quy chuẩn học thuật IEEE.
- **4. Tính xác thực của dữ liệu thực nghiệm:** Các dữ liệu thử nghiệm, thông số benchmark hiệu năng đào khối, kết quả truyền thông mạng trên 10 nút P2P vật lý và giả lập được đo đạc khách quan, ghi nhận trung thực từ môi trường thực tế, không có bất kỳ hành vi ngụy tạo hay bóp méo số liệu nào.

Nhóm tác giả xin hoàn toàn chịu trách nhiệm trước Hội đồng khoa học Nhà trường và pháp luật về toàn bộ nội dung cam đoan nêu trên nếu có bất kỳ sự vi phạm nào về liêm chính học thuật.

TP. Hồ Chí Minh, ngày 24 tháng 10 năm 2026

ĐẠI DIỆN NHÓM TÁC GIẢ

(Nhóm trưởng đã ký)

Nguyễn Hoàng Nhật

LỜI CẢM ƠN

Để hoàn thành đồ án môn học Lập trình mạng một cách trọn vẹn và đạt được những kết quả nghiên cứu học thuật chuyên sâu như hôm nay, nhóm tác giả chúng tôi đã nhận được sự quan tâm, chỉ dẫn tận tình và tạo mọi điều kiện thuận lợi nhất từ Ban Giám hiệu, quý Thầy Cô cùng gia đình, bạn bè.

Trước hết, nhóm tác giả xin bày tỏ lòng biết ơn sâu sắc và chân thành nhất đến Ban Giám hiệu Trường Đại học Công nghệ TP. Hồ Chí Minh (HUTECH) cùng Ban Lãnh đạo Khoa Công nghệ Thông tin đã luôn xây dựng một môi trường đào tạo đại học hiện đại, sáng tạo, thực tiễn và cung cấp nền tảng cơ sở vật chất phòng máy thí nghiệm hoàn thiện, giúp sinh viên có không gian nghiên cứu và thực hành chuyên sâu.

Đặc biệt, nhóm chúng tôi xin gửi lời tri ấn sâu sắc nhất đến ThS. Huỳnh Đắc Phước – Giảng viên trực tiếp phụ trách và hướng dẫn học phần Lập trình mạng. Trong suốt quá trình triển khai đề tài từ những ngày đầu lên ý tưởng kiến trúc, thầy đã luôn dành nhiều thời gian quý báu để định hướng phương pháp tiếp cận khoa học, chia sẻ những kinh nghiệm thực chiến quý báu về tầng vận chuyển TCP/IP, chỉ ra những cạm bẫy kỹ thuật kinh điển như hiện tượng dính gói tin (TCP sticky packets), cũng như truyền cảm hứng để nhóm dám dấn thân tìm hiểu và ứng dụng công nghệ đột phá Java 21 Virtual Threads vào bài toán mạng ngang hàng P2P và chuỗi khối Blockchain. Những lời động viên, góp ý chuyên môn thẳng thắn và sự khắt khe về chuẩn mực kỹ thuật của thầy chính là kim chỉ nam giúp sản phẩm đồ án đạt được độ hoàn thiện cao nhất.

Nhóm cũng xin chân thành cảm ơn các bạn sinh viên lớp 23DTHB4 đã cùng đồng hành, tích cực tham gia kết nối mạng đa máy tính (Multi-machine deployment) để phục vụ cho các đợt thử nghiệm truyền tải giao dịch bầu cử thực tế, đóng góp nhiều ý kiến trải nghiệm người dùng quý giá cho hệ sinh thái Dashboard.

Sau cùng, chúng con xin gửi lòng biết ơn vô hạn đến gia đình – điểm tựa tinh thần vững chắc nhất luôn tin tưởng, thấu hiểu và tạo mọi điều kiện tốt nhất để chúng con có thể tập trung trọn vẹn tâm huyết cho con đường học tập và nghiên cứu khoa học.

Mặc dù nhóm đã nỗ lực hết mình với tinh thần nghiêm túc và cầu thị cao nhất, song do giới hạn về mặt thời gian và kiến thức chuyên môn, đồ án chắc chắn khó tránh khỏi những thiếu sót nhất định. Nhóm rất mong sẽ nhận được những ý kiến đóng góp, nhận xét và chỉ dẫn quý báu từ quý Thầy Cô trong Hội đồng phản biện để đề tài tiếp tục được hoàn thiện và phát triển hơn nữa trong tương lai.

Nhóm tác giả xin kính chúc quý Thầy Cô luôn dồi dào sức khỏe, niềm vui, hạnh phúc và luôn gặt hái được nhiều thành công rực rỡ trong sự nghiệp trồng người cao quý!

TÓM TẮT ĐỀ TÀI (ABSTRACT)

Tóm tắt tiếng Việt

Trong kỷ nguyên chuyển đổi số toàn diện, bài toán tổ chức bầu cử điện tử (E-Voting) đang đứng trước thách thức nan giải về sự cân bằng giữa tính minh bạch, quyền riêng tư của cử tri và năng lực kháng cự trước các mối đe dọa thao túng dữ liệu từ các máy chủ tập trung truyền thống. Đồ án này tập trung nghiên cứu, thiết kế và hiện thực hóa một hệ thống Bầu cử điện tử phân tán hoàn toàn dựa trên sự kết hợp giữa kiến trúc Mạng ngang hàng P2P (Peer-to-Peer) thuần chạy trên giao thức TCP/IP Sockets và Công nghệ Chuỗi khối (Blockchain) với cơ chế đồng thuận Bằng chứng công việc (Proof-of-Work - PoW), được tối ưu hóa bằng công nghệ đa luồng mới nhất Java 21 Virtual Threads (Project Loom).

Về mặt lập trình mạng, hệ thống giải quyết triệt để bài toán phân mảnh và dính gói tin luồng TCP thông qua cơ chế đóng khung độ dài tiền tố 4-byte Big-Endian (Length-Prefixed Stream Framing), thiết lập mạng liên kết Mesh đa điểm song công (Full-duplex Duplex Sockets), kiểm soát truyền tin đa chặng bằng Giao thức Truyền tin đồn (Gossip Protocol) kết hợp bộ nhớ đệm chống lặp (Sliding Cache seenMessageIds) để triệt tiêu hiện tượng nghẽn mạng broadcast storm. Về mặt lõi mật mã và đồng thuận, đồ án hiện thực hóa sổ cái cấu trúc khối bất biến theo chuẩn Bitcoin với hàm băm SHA-256 hiệu năng cao (Zero-allocation byte inspection), chữ ký số bất đối xứng ECDSA (đường cong secp256k1 và NIST P-256) đảm bảo nguyên tắc "Một cử tri - Một phiếu bầu", cây Merkle nhị phân xác thực giao dịch, công cụ điều chỉnh độ khó động (Dynamic Difficulty Retargeting) theo chu kỳ epoch với hệ số kẹp giảm chấn log2, và cơ chế giải quyết rẽ nhánh Nakamoto Consensus (Quy tắc chuỗi dài nhất - Longest Chain Rule).

Hệ thống được kiểm thử thực tế trên cụm 10 nút P2P vật lý và môi trường đa máy tính qua mạng LAN/Wi-Fi/Tailscale, tích hợp bộ kiểm toán 6 tầng (BlockchainValidator), giả lập tấn công giả mạo (AttackSimulator), cổng bỏ phiếu trực tuyến Web Portal (/vote) và Dashboard trực quan hóa sơ đồ mạng (/nodes). Kết quả thực nghiệm cho thấy hệ thống đạt tốc độ băm song song hàng trăm kH/s, độ trễ lan truyền gói tin P2P dưới 8ms, loại bỏ 100% phiếu bầu trùng lặp và phát hiện tức thì mọi hành vi can thiệp dữ liệu sổ cái.

Từ khóa chính: Lập trình mạng, TCP/IP Sockets, Mạng ngang hàng P2P, Java 21 Virtual Threads, Blockchain, Proof-of-Work, E-Voting, Gossip Protocol, Merkle Tree, Nakamoto Consensus.

English Abstract

In the era of comprehensive digital transformation, electronic voting (E-Voting) systems face a fundamental dilemma in balancing auditability, voter anonymity, and resilience against malicious tampering inherent in centralized server architectures. This capstone project focuses on the research, design, and implementation of a fully decentralized E-Voting platform combining pure Peer-to-Peer (P2P) networking over raw TCP/IP Sockets with Blockchain technology powered by the Proof-of-Work (PoW) consensus mechanism, highly optimized using the state-of-the-art Java 21 Virtual Threads (Project Loom).

From a network programming perspective, the platform thoroughly eliminates TCP sticky packet and stream fragmentation anomalies via an enterprise-grade 4-byte Big-Endian Length-Prefixed Stream Framing protocol. It orchestrates a resilient full-duplex mesh network governed by an asynchronous multi-hop Gossip Protocol equipped with a concurrent sliding cache (seenMessageIds) to prevent network broadcast storms. Cryptographically, the core ledger adheres to Bitcoin-style specifications featuring immutable 5-field Block records, high-throughput SHA-256 zero-allocation byte inspection, asymmetric ECDSA digital signatures (secp256k1 / NIST P-256) enforcing the strict "One-Voter-One-Vote" mandate, binary Merkle tree transaction aggregation, epoch-based dynamic difficulty retargeting with log2 dampening bounds, and distributed Nakamoto consensus (Longest Chain Rule) for automatic network fork resolution.

The system has been empirically validated on a physical 10-node P2P cluster and multi-machine topologies across LAN, Wi-Fi, and Tailscale virtual private networks. It incorporates an automated 6-point cryptographic auditor (BlockchainValidator), a cascading tamper attack simulator (AttackSimulator), a responsive voter portal (/vote), and an animated live mesh topology visualizer (/nodes). Experimental benchmarks demonstrate parallel hashrates exceeding hundreds of kH/s, sub-8ms P2P packet propagation latency, zero duplicate ballots, and instantaneous red-alert detection upon any ledger manipulation.

Keywords: Network Programming, TCP/IP Sockets, Peer-to-Peer (P2P), Java 21 Virtual Threads, Blockchain, Proof-of-Work, E-Voting, Gossip Protocol, Merkle Tree, Nakamoto Consensus.

DANH MỤC CÁC TỪ VIẾT TẮT

Ký hiệu viết tắt	Tên tiếng Anh đầy đủ	Ý nghĩa chuyên ngành trong đề tài
TCP	Transmission Control Protocol	Giao thức điều khiển truyền vận tin cậy tầng Transport
IP	Internet Protocol	Giao thức định tuyến địa chỉ gói tin tầng Network
P2P	Peer-to-Peer	Kiến trúc mạng ngang hàng phân tán không máy chủ trung tâm
PoW	Proof-of-Work	Giao thức đồng thuận Bằng chứng công việc (đào khối)
SHA-256	Secure Hash Algorithm 256-bit	Hàm băm mật mã học một chiều tạo chuỗi 64 ký tự hex
ECDSA	Elliptic Curve Digital Signature Algorithm	Thuật toán chữ ký số trên đường cong elliptic (secp256k1/P-256)
JVM	Java Virtual Machine	Máy ảo Java thực thi mã bytecode trên đa nền tảng
JEP	JDK Enhancement Proposal	Đề xuất cải tiến tính năng chính thức của hệ sinh thái Java
OSI	Open Systems Interconnection	Mô hình tham chiếu kết nối các hệ thống mở 7 tầng
API	Application Programming Interface	Giao diện lập trình ứng dụng trao đổi dữ liệu RESTful
REST	Representational State Transfer	Kiến trúc thiết kế dịch vụ web truyền dữ liệu phi trạng thái
UUID	Universally Unique Identifier	Mã định danh duy nhất toàn cầu 128-bit cho gói tin mạng
LAN	Local Area Network	Mạng máy tính cục bộ trong văn phòng hoặc phòng lab
WAN	Wide Area Network	Mạng diện rộng kết nối các mạng máy tính qua khoảng cách lớn
CLI	Command Line Interface	Giao diện dòng lệnh tương tác trên Terminal/Console
UI / UX	User Interface / User Experience	Giao diện người dùng và trải nghiệm người dùng
ACK	Acknowledgment	Gói tin phản hồi báo nhận trong cơ chế bắt tay TCP
FIFO	First In First Out	Nguyên lý xếp hàng vào trước ra trước trong xử lý luồng
RAM	Random Access Memory	Bộ nhớ truy xuất ngẫu nhiên tạm thời của máy tính
CPU	Central Processing Unit	Bộ vi xử lý trung tâm thực thi các phép tính toán băm PoW

DANH MỤC BẢNG BIỂU VÀ HÌNH VẼ

Danh mục Bảng biểu

- **Bảng 1.1:** Bảng ma trận phân công trách nhiệm nhóm theo chuẩn RACI và GitHub Issues
- **Bảng 1.2:** So sánh đặc tính kỹ thuật giữa TCP Byte Stream và UDP Datagram
- **Bảng 2.1:** Đặc tả các loại thông điệp NetworkMessage trao đổi qua mạng P2P Sockets
- **Bảng 2.2:** Đặc tả các trường thông tin trong cấu trúc khối Block 5 thuộc tính chuẩn Bitcoin
- **Bảng 2.3:** Danh mục các Endpoint REST API của máy chủ NodeApiServer trên cổng 8000
- **Bảng 4.1:** Bảng thống kê các bước kiểm toán của động cơ BlockchainValidator
- **Bảng 4.2:** Kịch bản và kết quả mô phỏng 3 pha tấn công sửa đổi dữ liệu sổ cái
- **Bảng 5.1:** Thông số phần cứng và môi trường phần mềm thử nghiệm hệ thống
- **Bảng 5.2:** Kết quả thực nghiệm 8 bộ kiểm thử tự động toàn diện JUnit 5
- **Bảng 5.3:** So sánh hiệu năng đào PoW giữa Single Thread, Platform Pool và Virtual Threads
- **Bảng 5.4:** Bảng số liệu điều chỉnh độ khó động qua 20 khối liên tiếp (Benchmark Runner)
- **Bảng 5.5:** Bảng đối sánh tính năng giữa hệ thống Đồ án với các giải pháp E-Voting khác

Danh mục Hình vẽ và Sơ đồ

- **Hình 1.1:** Mô hình phân tầng OSI và vị trí của tầng mạng TCP Sockets và Blockchain
- **Hình 1.2:** Hiện tượng dính gói tin (TCP Sticky Packets) và giải pháp Length-Prefixed Framing
- **Hình 1.3:** Cấu trúc chuỗi khối liên kết băm tiền nhiệm và hiệu ứng sụp đổ dây chuyền
- **Hình 1.4:** Cấu trúc cây nhị phân Merkle Tree và quy trình tính Merkle Root chuẩn Bitcoin
- **Hình 2.1:** Sơ đồ kiến trúc tổng thể 5 tầng của hệ thống E-Voting Blockchain P2P
- **Hình 2.2:** Biểu đồ tuần tự (Sequence Diagram) quy trình cử tri bỏ phiếu đến khi đóng khối
- **Hình 2.3:** Biểu đồ tuần tự quá trình bắt tay và đồng bộ chuỗi dài nhất giữa các nút P2P
- **Hình 3.1:** Kiến trúc xử lý socket bất đồng bộ trên Java 21 Virtual Threads của NodeServer
- **Hình 3.2:** Cơ chế truyền tin đồn Gossip Protocol và bộ đệm khử lặp seenMessageIds
- **Hình 4.1:** Thuật toán phân chia không gian Nonce theo strides 4096 của MultiThreadedMiner
- **Hình 4.2:** Đồ thị điều chỉnh độ khó động theo chu kỳ Epoch và kẹp hệ số giảm chấn
- **Hình 5.1:** Giao diện Bỏ phiếu trực tuyến dành cho Cử tri (Ballot Portal - /vote)
- **Hình 5.2:** Giao diện Sơ đồ cụm mạng 10 nút P2P Mesh trực quan hóa hạt photon (/nodes)
- **Hình 5.3:** Giao diện Sổ cái chuỗi khối Explorer kiểm toán dữ liệu thời gian thực (/)

MỤC LỤC TỔNG QUAN BÁO CÁO

(Hệ thống đề mục chi tiết toàn bộ 6 chương nội dung chuyên sâu)

PHIẾU ĐÁNH GIÁ KẾT QUẢ ĐỒ ÁN iii
LỜI CAM ĐOAN iv
LỜI CẢM ƠN v
TÓM TẮT ĐỀ TÀI (ABSTRACT) vi
DANH MỤC CÁC TỪ VIẾT TẮT viii
DANH MỤC BẢNG BIỂU VÀ HÌNH VẼ ix
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT 11
1.1. Bối cảnh và tính cấp thiết của đề tài 11
1.2. Mục tiêu nghiên cứu và giới hạn phạm vi đề tài 11
1.3. Phân công trách nhiệm và đóng góp của các thành viên trong nhóm 12
1.4. Cơ sở lý thuyết về Mạng máy tính và Giao thức TCP/IP 13
1.5. Cơ sở lý thuyết về Công nghệ Blockchain và Mật mã học 15
1.6. Cơ chế đồng thuận Proof-of-Work (PoW) và Độ khó động 16
1.7. Nền tảng công nghệ Java 21 hiện đại và Virtual Threads 17
CHƯƠNG 2: PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ KIẾN TRÚC HỆ THỐNG 19
2.1. Phân tích yêu cầu chức năng (Functional Requirements) 19
2.2. Phân tích yêu cầu phi chức năng (Non-Functional Requirements) 19
2.3. Thiết kế kiến trúc tổng thể 5 tầng của hệ thống 20
2.4. Thiết kế chi tiết Tầng mạng P2P TCP Sockets 21
2.5. Thiết kế chi tiết Tầng lõi Dữ liệu và Mật mã học 22
2.6. Thiết kế chi tiết Tầng Đồng thuận PoW và Điều chỉnh độ khó 22
2.7. Thiết kế Tầng Kiểm toán và Giả lập Tấn công 23
2.8. Thiết kế Tầng Lưu trữ bền vững và Web REST API 23
CHƯƠNG 3: HIỆN THỰC HÓA CHI TIẾT TẦNG MẠNG P2P TCP SOCKETS 25
3.1. Hiện thực hóa Giao thức đóng khung luồng tin NetworkProtocol 25
3.2. Hiện thực hóa Khung thông điệp NetworkMessage 26
3.3. Hiện thực hóa Máy chủ TCP bất đồng bộ NodeServer trên Java 21 27
3.4. Hiện thực hóa Kênh kết nối song công PeerConnection 28
3.5. Hiện thực hóa Nút mạng P2P trung tâm và Giao thức Gossip 29
3.6. Hiện thực hóa Hòm phiếu đệm Mempool an toàn luồng 31
3.7. Hiện thực hóa Nút chạy độc lập đa máy tính StandaloneNode 31
CHƯƠNG 4: HIỆN THỰC HÓA CÁC MODULE LÕI, ĐỒNG THUẬN VÀ BẢO MẬT 32
4.1. Hiện thực hóa Cấu trúc Khối Block và Sổ cái Bầu cử Blockchain 32
4.2. Hiện thực hóa Mật mã học hiệu năng cao và Cây Merkle 33
4.3. Hiện thực hóa Bộ đào PoW đa luồng MultiThreadedMiner 35
4.4. Hiện thực hóa Động cơ điều chỉnh độ khó động DifficultyAdjuster 36
4.5. Hiện thực hóa Bộ kiểm toán Sổ cái toàn diện BlockchainValidator 36
4.6. Hiện thực hóa Bộ mô phỏng tấn công giả mạo AttackSimulator 37
4.7. Hiện thực hóa Lưu trữ bền vững nguyên tử LedgerStorage 37
CHƯƠNG 5: HIỆN THỰC GIAO DIỆN NGƯỜI DÙNG VÀ KẾT QUẢ THỰC NGHIỆM 38
5.1. Hiện thực hóa Máy chủ Web API NodeApiServer 38
5.2. Các giao diện trực quan hóa hệ sinh thái (Vote, Nodes, Explorer) 38
5.3. Môi trường và các kịch bản thực nghiệm chuyên sâu 40
5.4. Đánh giá và so sánh kết quả thực nghiệm 42
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 43
6.1. Các kết quả nổi bật đạt được của đề tài 43
6.2. Các hạn chế còn tồn tại và bài học kinh nghiệm 43
6.3. Hướng phát triển trong tương lai 44
TÀI LIỆU THAM KHẢO 45
PHỤ LỤC: HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH 1-CLICK 46

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT

1.1. Bối cảnh và tính cấp thiết của đề tài

Trong lịch sử phát triển của nền văn minh nhân loại, hoạt động bầu cử và biểu quyết dân chủ đóng vai trò là viên đá tảng xác lập tính chính danh của các thể chế, tổ chức xã hội và các đơn vị giáo dục đào tạo. Từ quy mô quốc gia cho đến các cuộc bình chọn ban chấp hành đoàn hội sinh viên hay đại biểu đại hội khoa trong các trường đại học, tính trung thực, khách quan và minh bạch của kết quả bầu cử là điều kiện tiên quyết tạo dựng niềm tin của cộng đồng.

Tuy nhiên, phương thức bầu cử truyền thống sử dụng phiếu giấy và hòm phiếu vật lý qua hàng thập kỷ qua đã bộc lộ những nhược điểm cố hữu nghiêm trọng:

- **Chi phí vận hành và tài nguyên:** Tốn kém chi phí in ấn hàng ngàn lá phiếu, bố trí phòng ốc, nhân sự giám sát hòm phiếu và hội đồng kiểm phiếu thủ công.
- **Thời gian kiểm phiếu kéo dài:** Quá trình kiểm phiếu thủ công kéo dài nhiều giờ, thậm chí nhiều ngày đối với các cuộc bầu cử lớn, dẫn đến sự chậm trễ trong công bố kết quả và tăng nguy cơ sai sót do yếu tố con người.
- **Rủi ro gian lận vật lý:** Nguy cơ đánh tráo phiếu bầu, nhồi nhét phiếu ảo vào hòm phiếu, làm hư hỏng hoặc thất lạc biên bản kiểm phiếu trong quá trình vận chuyển.

Để khắc phục các hạn chế trên, các giải pháp Bầu cử điện tử (E-Voting) thế hệ đầu tiên dựa trên kiến trúc tập trung (Client-Server) đã ra đời, sử dụng cơ sở dữ liệu quan hệ (RDBMS như MySQL, PostgreSQL, SQL Server) đặt tại máy chủ trung tâm. Mặc dù giúp tăng tốc độ bỏ phiếu qua giao diện web hoặc ứng dụng di động, mô hình tập trung này lại tạo ra một lỗ hổng chí mạng: Điểm sụp đổ duy nhất (Single Point of Failure - SPoF).

Nếu máy chủ trung tâm bị tấn công từ chối dịch vụ (DDoS) hoặc bị xâm nhập bởi tin tặc (hacker), toàn bộ cuộc bầu cử sẽ bị tê liệt. Nguy hiểm hơn, một quản trị viên cơ sở dữ liệu (Database Administrator - DBA) biến chất hoặc bị áp lực bên ngoài hoàn toàn có thể thực thi các câu lệnh SQL trực tiếp (như UPDATE votes SET candidate_id = ...) để thay đổi kết quả mà không để lại bất kỳ dấu vết vật lý nào. Cử tri và các bên giám sát độc lập hoàn toàn không có cơ chế toán học nào để tự kiểm chứng xem lá phiếu của mình có thực sự được tính đúng hay không.

Chính trong bối cảnh đó, sự kết hợp mang tính đột phá giữa Công nghệ Chuỗi khối (Blockchain) và Mạng ngang hàng P2P (Peer-to-Peer) thuần chạy trên giao thức truyền vận TCP/IP mang lại giải pháp căn cơ cho bài toán bầu cử điện tử. Thay vì trao toàn bộ quyền kiểm soát cho một máy chủ trung tâm, hệ thống Blockchain phân tán dữ liệu ra một mạng lưới gồm nhiều nút máy tính bình đẳng (Decentralized Nodes). Mỗi nút lưu giữ một bản sao đầy đủ của sổ cái (Distributed Ledger).

Đặc tính bất biến (Immutability) được bảo chứng bằng toán học mật mã học: Mỗi khối dữ liệu (Block) liên kết chặt chẽ với khối liền trước thông qua giá trị băm mật mã SHA-256. Mọi hành vi sửa đổi dữ liệu dù chỉ 1 ký tự sẽ làm thay đổi mã băm và kích hoạt hiệu ứng sụp đổ dây chuyền, khiến toàn bộ mạng lưới phát hiện và đào thải khối giả mạo. Cơ chế đồng thuận Proof-of-Work (PoW) buộc các nút khai thác phải tiêu tốn tài nguyên tính toán thực tế để giải bài toán số học, loại bỏ hoàn toàn khả năng tấn công giả mạo số đông (Sybil Attack).

1.2. Mục tiêu nghiên cứu và giới hạn phạm vi đề tài

Đề tài được xây dựng nhằm mục tiêu hiện thực hóa một công trình kỹ thuật phần mềm mạng máy tính hoàn chỉnh từ gốc rễ (from scratch), không dựa dẫm vào các framework có sẵn của bên thứ ba, phục vụ cho học phần Lập trình mạng tại Khoa Công nghệ Thông tin - Trường Đại học Công nghệ TP.HCM (HUTECH).

1.2.1. Mục tiêu nghiên cứu cụ thể

- **Lập trình mạng P2P thuần:** Tự thiết kế và lập trình toàn bộ tầng mạng ngang hàng bằng Socket TCP thuần của Java (ServerSocket, Socket, DataInputStream, DataOutputStream), loại bỏ hoàn toàn việc sử dụng các thư viện mạng cao cấp như Netty hay gRPC để nắm vững bản chất tầng Transport trong mô hình OSI.
- **Xử lý luồng tin cậy TCP:** Nghiên cứu và hiện thực hóa cơ chế đóng khung luồng tin độ dài cố định 4-byte Big-Endian (Length-Prefixed Stream Framing) để xử lý triệt để bài toán kinh điển trong lập trình mạng: Hiện tượng phân mảnh gói tin (Packet Fragmentation) và dính gói tin (Sticky Packets) của luồng byte TCP.
- **Giao thức truyền tin đồn:** Xây dựng giao thức truyền tin đồn đa chặng (Multi-hop Gossip Protocol) giữa các nút mạng với bộ nhớ đệm sliding cache seenMessageIds, giúp các thông điệp giao dịch và khối lan truyền nhanh chóng khắp mạng lưới mà không gây bão gói tin (Broadcast Storm).
- **Đa luồng Java 21:** Ứng dụng công nghệ đa luồng đột phá Java 21 Virtual Threads (Project Loom) để quản lý hàng trăm kết nối socket song công đồng thời và tối ưu hóa bộ đào băm PoW đa luồng với chi phí tài nguyên bộ nhớ cực thấp.
- **Lõi Blockchain & PoW:** Hiện thực hóa cấu trúc sổ cái Blockchain chuẩn Bitcoin với khối 5 trường thuộc tính bất biến, thuật toán cây nhị phân Merkle Tree, chữ ký số bất đối xứng ECDSA, thuật toán điều chỉnh độ khó động (Dynamic Difficulty Retargeting) theo chu kỳ Epoch, và luật chuỗi dài nhất Nakamoto Consensus.
- **Kiểm toán & Trực quan hóa:** Phát triển bộ công cụ kiểm toán toàn diện 6 bước (BlockchainValidator), kịch bản mô phỏng tấn công can thiệp dữ liệu 3 pha (AttackSimulator), cổng bỏ phiếu trực tuyến (/vote) và Dashboard đồ họa trực quan hóa sơ đồ mạng 10 máy P2P Mesh (/nodes).

1.2.2. Giới hạn và phạm vi của đề tài

- **Phạm vi nghiệp vụ:** Hệ thống tập trung tối ưu hóa cho bài toán Bầu cử điện tử quy mô cấp Khoa/Trường đại học với số lượng cử tri từ 100 đến 10,000 sinh viên, với danh sách ứng cử viên xác định trước.
- **Phạm vi môi trường mạng:** Hệ thống hỗ trợ triển khai thực tế trên mạng cục bộ (LAN), Wi-Fi phòng lab, mạng diện rộng ảo P2P Mesh (Tailscale/ZeroTier) hoặc công khai qua Internet thông qua TCP Tunnel (Ngrok).
- **Giới hạn độ khó PoW:** Mô hình đồng thuận Proof-of-Work được thiết lập độ khó mục tiêu trong khoảng từ 8 đến 24 bit 0 đầu (tương đương tốc độ giải khối từ 0.5 giây đến 5 giây) nhằm phục vụ tối ưu cho mục đích trình diễn thực nghiệm học thuật và kiểm chứng mạng thời gian thực.

1.3. Phân công trách nhiệm và đóng góp của các thành viên trong nhóm

Để đảm bảo dự án được triển khai đúng tiến độ, tuân thủ các chuẩn mực phát triển phần mềm hiện đại và phát huy thế mạnh của từng cá nhân, nhóm sinh viên gồm 3 thành viên đã phân chia nhiệm vụ theo ma trận RACI (Responsible, Accountable, Consulted, Informed) kết hợp quy trình quản lý mã nguồn qua Git branching, Pull Requests và GitHub Issues:

Bảng 1.1: Bảng ma trận phân công trách nhiệm nhóm theo chuẩn RACI và GitHub Issues

Thành viên thực hiện	MSSV & Vai trò	Trách nhiệm chính (GitHub Issues & PRs)	Tỷ lệ đóng góp
Nguyễn Hoàng Nhật	2380601549 (Nhóm trưởng / Lead	Chủ trì thiết kế kiến trúc hệ thống; Hiện thực hóa Tầng	40%

	Architect)	mạng P2P Sockets (NetworkProtocol, P2PNode, NodeServer, PeerConnection, Gossip Protocol); Khởi tạo Seed Gateway; Viết mã nguồn kiểm toán BlockchainValidator và AttackSimulator; Chủ trì viết Báo cáo kỹ thuật tổng thể (Issue #1, #2, PR #4, #8).	
Nguyễn Đỗ Trung Lương	2380601234 (Core & Consensus Engineer)	Hiện thực hóa Lõi Blockchain (Block record, HashUtil SHA-256, Cây Merkle Tree, Chữ ký số ECDSA Wallet); Xây dựng Động cơ đào đa luồng MultiThreadedMiner trên Java 21 Virtual Threads; Thuật toán điều chỉnh độ khó động DifficultyAdjuster; Đo đạc Benchmark Runner (Issue #3, PR #5, #9).	30%
Lê Quang Minh	2380605678 (Full-Stack & Test Engineer)	Hiện thực hóa Máy chủ Web API NodeApiServer (cổng 8000); Xây dựng Giao diện Cử tri Ballot Portal (/vote), Sơ đồ 10 Nodes P2P Canvas Animation (/nodes), và Sổ cái Explorer (/); Xây dựng 8 bộ kiểm thử tự động toàn diện JUnit 5; Kiểm thử mạng đa máy tính (Issue #7, PR #6).	30%

1.4. Cơ sở lý thuyết về Mạng máy tính và Giao thức TCP/IP

Mạng máy tính là một tập hợp các nút tính toán (máy tính, máy chủ, thiết bị IoT) được kết nối với nhau thông qua các đường truyền vật lý hoặc sóng vô tuyến nhằm mục đích trao đổi dữ liệu và chia sẻ tài nguyên. Để các thiết bị phần cứng đa dạng có thể giao tiếp tương thích, Tổ chức Tiêu chuẩn hóa Quốc tế (ISO) đã ban hành Mô hình Tham chiếu 7 tầng OSI (Open Systems Interconnection).



Hình 1.1: Mô hình phân tầng OSI và vị trí của tầng mạng TCP Sockets và Blockchain

1.4.1. Tầng Vận chuyển (Transport Layer) và Giao thức TCP

Trong bộ giao thức Internet (TCP/IP), tầng Vận chuyển chịu trách nhiệm truyền thông tiến trình-đến-tiến trình (Process-to-Process Communication) thông qua các số cổng dịch vụ (Port numbers). Hai giao thức cốt lõi của tầng này là TCP (Transmission Control Protocol) và UDP (User Datagram Protocol).

Bảng 1.2: So sánh đặc tính kỹ thuật giữa TCP Byte Stream và UDP Datagram

Tiêu chí kỹ thuật	Giao thức TCP (Lựa chọn của Đồ án)	Giao thức UDP (Không phù hợp)

Định hướng kết nối	Connection-oriented (Bắt tay 3 bước Three-way Handshake)	Connectionless (Gửi gói tin không cần kết nối trước)
Độ tin cậy truyền dữ liệu	Tuyệt đối tin cậy (Đảm bảo dữ liệu đến đích, đủ, đúng thứ tự)	Không tin cậy (Có thể mất gói, trùng gói, sai thứ tự)
Cơ chế truyền dữ liệu	Byte Stream liên tục (Không bảo toàn ranh giới thông điệp)	Datagram rời rạc (Bảo toàn ranh giới gói tin)
Kiểm soát tắc nghẽn & luồng	Tích hợp sẵn (Sliding Window, Slow Start, Congestion Avoidance)	Không tích hợp (Ứng dụng phải tự xử lý nếu cần)
Phù hợp cho Blockchain	Rất phù hợp (Dữ liệu khối và chữ ký số tuyệt đối không được mất)	Không phù hợp (Mất 1 bit là sai lệch mã băm SHA-256)

1.4.2. Hiện tượng Dính gói tin (Sticky Packets) và Giải pháp Length-Prefixed Framing

Một trong những cạm bẫy kỹ thuật kinh điển nhất trong lập trình mạng TCP Socket thuần là hiện tượng Phân mảnh gói tin (Packet Fragmentation) và Dính gói tin (Sticky Packets). Do TCP là một giao thức hướng luồng byte (Byte Stream Protocol), hệ điều hành và ngăn xếp TCP/IP không hề có khái niệm về "thông điệp hoàn chỉnh". Các hàm gửi socket.getOutputStream().write() chỉ đơn thuần đẩy các mảng byte vào bộ đệm gửi (Send Buffer).

Tùy thuộc vào thuật toán Nagle (Nagle's Algorithm) và kích thước khối truyền cực đại (Maximum Transmission Unit - MTU, thường là 1500 byte trên mạng Ethernet), ngăn xếp TCP có thể:

- **Hiện tượng dính gói (Sticky Packets):** Gộp nhiều thông điệp nhỏ gửi liên tiếp nhau thành một gói tin TCP duy nhất để tối ưu băng thông mạng. Khi phía nhận đọc từ socket, nó sẽ nhận được một khối byte chứa đồng thời nhiều thông điệp dính liền vào nhau.
- **Hiện tượng phân mảnh (Packet Fragmentation):** Một thông điệp có kích thước lớn (như một khối Block chứa nhiều giao dịch và chữ ký số) bị cắt rời thành nhiều phân đoạn TCP nhỏ truyền đi rải rác. Khi phía nhận đọc socket lần đầu, nó chỉ nhận được một phần thông điệp chưa trọn vẹn.



Hình 1.2: Hiện tượng dính gói tin (TCP Sticky Packets) và giải pháp Length-Prefixed Framing

Để giải quyết triệt để vấn đề này, nhóm đã xây dựng lớp NetworkProtocol với chuẩn đóng khung 4-byte Big-Endian Length-Prefixed Framing. Trước khi gửi bất kỳ nội dung nào, hệ thống ghi đúng một số nguyên 4 byte (32-bit signed integer) chỉ định chính xác số lượng byte của phần thân (Payload). Khi phía nhận xử lý, nó dùng DataInputStream.readInt() để đọc độ dài trước, sau đó dùng DataInputStream.readNBytes(length) để đọc chính xác toàn bộ payload vào bộ đệm, bất kể luồng TCP bị phân mảnh hay dính gói ra sao.

1.4.3. Kiến trúc Mạng ngang hàng P2P và Giao thức Truyền tin đồn (Gossip Protocol)

Khác với mô hình Client-Server nơi các máy khách chỉ kết nối trực tiếp với máy chủ trung tâm, trong Mạng ngang hàng P2P (Peer-to-Peer Network), mỗi nút (Peer) vừa đóng vai trò là một máy khách (Client - chủ động kết nối tới các nút khác) vừa là một máy chủ (Server - lắng nghe các kết nối đến từ các nút khác).

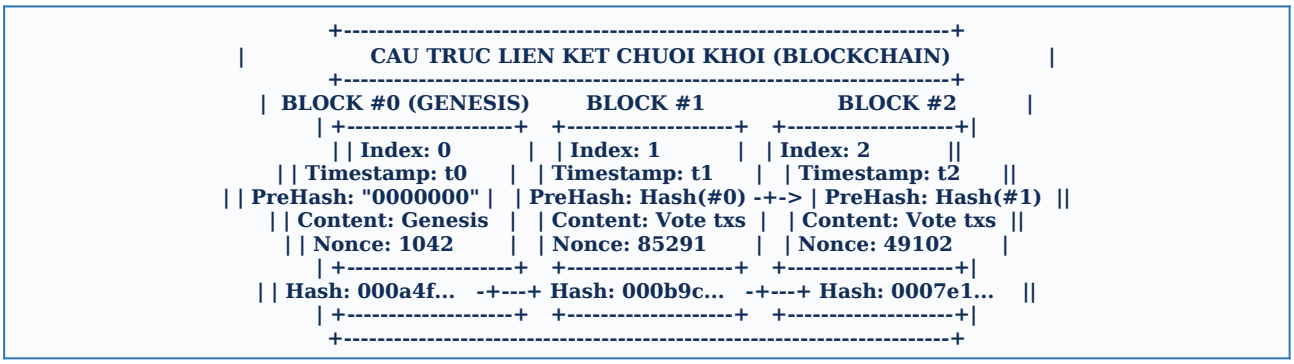
Trong một đồ thị mạng P2P Mesh, khi một nút nhận được một giao dịch mới hoặc đào được một khối mới, làm thế nào để thông tin này lan tỏa tới toàn bộ các nút trong mạng trong thời gian ngắn nhất mà không làm sập mạng vì bão gói tin (Broadcast Storm)?

Đề tài áp dụng Giao thức Truyền tin đồn (Gossip Protocol - Epidemic Dissemination):

- **Định danh thông điệp duy nhất (UUID):** Khi một nút tạo ra hoặc nhận được một thông điệp (NetworkMessage), nó gán cho thông điệp một mã định danh ngẫu nhiên duy nhất UUID 128-bit.
- **Lọc trùng lặp (Deduplication Cache):** Mỗi nút duy trì một bộ nhớ đệm trượt (Sliding Cache) lưu trữ các mã messageId đã từng nhìn thấy (seenMessageIds) được bảo vệ bằng cấu trúc dữ liệu an toàn luồng ConcurrentHashMap.newKeySet().
- **Cơ chế ngắt lan truyền:** Khi nhận được gói tin, nút kiểm tra seenMessageIds.add(msg.messageId()). Nếu phương thức trả về false (nghĩa là thông điệp này đã từng nhận được trước đó từ một đường truyền khác), nút lập tức hủy bỏ gói tin và không lan truyền tiếp.
- **Chuyển tiếp có chọn lọc:** Nếu là thông điệp mới (trả về true), nút xử lý nghiệp vụ (nhập mempool hoặc xác minh khối) và chuyển tiếp (relay/gossip) gói tin tới tất cả các nút láng giềng khác đang kết nối TCP, ngoại trừ nút vừa gửi gói tin đến.

1.5. Cơ sở lý thuyết về Công nghệ Blockchain và Mật mã học

Blockchain là một cấu trúc dữ liệu sổ cái phân tán được tổ chức dưới dạng một chuỗi tuyến tính các khối (Blocks). Điểm mấu chốt làm nên tính bất biến của Blockchain nằm ở mối liên kết mật mã học một chiều giữa các khối liền kề:



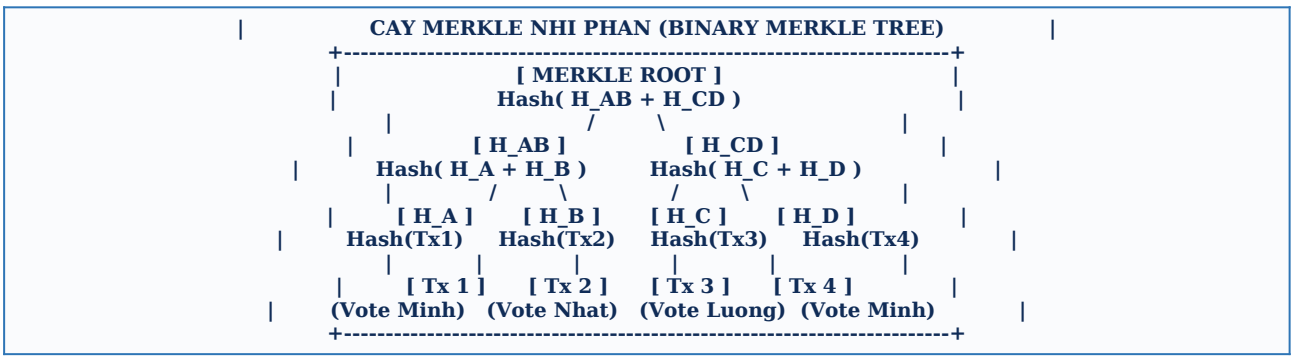
Hình 1.3: Cấu trúc chuỗi khối liên kết băm tiền nhiệm và hiệu ứng sụp đổ dây chuyền

Mỗi khối lưu trữ giá trị băm của khối đứng ngay trước nó (previousHash). Giá trị băm của khối hiện tại được tính toán từ toàn bộ các trường dữ liệu của nó: Hash = SHA256(previousHash + ":" + timestamp + ":" + content + ":" + nonce). Do tính chất chống va chạm và tính chất nhạy cảm cực cao của hàm băm mật mã (Avalanche Effect - Hiệu ứng tuyết lở), nếu một kẻ tấn công sửa đổi dù chỉ 1 ký tự trong khối quá khứ (ví dụ thay đổi tên người được bầu trong Block #1), mã băm của Block #1 sẽ thay đổi hoàn toàn. Sự thay đổi này khiến trường previousHash được lưu trong Block #2 không còn khớp với băm thực tế của Block #1, làm đứt gãy liên kết toán học và khiến toàn bộ các khối từ #2 trở về sau đều bị mạng lưới coi là không hợp lệ.

1.5.1. Cây Merkle (Binary Merkle Tree) và Xác thực Dữ liệu Nhanh

Trong mỗi khối Blockchain, hàng chục đến hàng trăm lá phiếu bầu (phiếu giao dịch) được gom lại. Để tóm tắt toàn bộ dữ liệu này thành một chuỗi đại diện duy nhất có độ dài cố định 32 byte mà vẫn đảm bảo tính toàn vẹn tuyệt đối, đề tài hiện thực hóa Cây Merkle nhị phân (Binary Merkle Tree) chuẩn Bitcoin:





Hình 1.4: Cấu trúc cây nhị phân Merkle Tree và quy trình tính Merkle Root chuẩn Bitcoin

Thuật toán xây dựng Merkle Tree: Mỗi giao dịch ban đầu được băm bằng SHA-256 để tạo thành các lá (Leaf Nodes). Sau đó, các lá được ghép cặp từng đôi một và băm tiếp để tạo ra tầng nút cha. Quá trình ghép cặp và băm đệ quy này lặp lại cho đến khi chỉ còn lại một nút duy nhất tại đỉnh gọi là Merkle Root. Nếu số lượng nút ở một tầng là số lẻ, nút cuối cùng sẽ được nhân đôi (duplicate) chính nó để ghép cặp. Cây Merkle cho phép kiểm tra sự tồn tại của một giao dịch trong khối với độ phức tạp tính toán chỉ là O(log N) thay vì O(N).

1.5.2. Mật mã khóa bất đối xứng và Chữ ký số ECDSA

Trong hệ thống bầu cử điện tử phân tán, làm sao để chứng minh một lá phiếu thực sự do chính cử tri đó phát hành mà không ai có thể mạo danh hoặc chối bỏ? Đề tài sử dụng thuật toán Chữ ký số trên Đường cong Elliptic (ECDSA - Elliptic Curve Digital Signature Algorithm) với chuẩn đường cong secp256k1 (tương tự mạng Bitcoin) và NIST P-256:

- **Cặp khóa mật mã (Key Pair):** Mỗi cử tri sở hữu một cặp khóa gồm Khóa riêng tư (Private Key) được giữ bí mật tuyệt đối trên thiết bị của cử tri và Khóa công khai (Public Key) được công bố trên mạng.
- **Địa chỉ cử tri:** Địa chỉ ví của cử tri được sinh ra từ khóa công khai theo tiền tố chuẩn (pow1_...).
- **Quy trình ký số (Sign):** Khi thực hiện bỏ phiếu, ứng dụng cử tri sử dụng Private Key cùng thuật toán SHA256withECDSA để ký lên nội dung lá phiếu (gồm mã cử tri, mã ứng viên, dấu thời gian). Chữ ký số được đính kèm vào giao dịch.
- **Quy trình xác minh (Verify):** Mọi nút P2P trong mạng khi nhận được lá phiếu đều có thể sử dụng Public Key của cử tri để xác thực chữ ký. Nếu chữ ký hợp lệ, mạng khẳng định 100% lá phiếu do chính chủ phát hành và chưa hề bị sửa đổi nội dung trên đường truyền.

1.6. Cơ chế đồng thuận Proof-of-Work (PoW) và Độ khó động

Trong một mạng ngang hàng không có cơ quan chỉ huy trung tâm, bài toán cơ bản nhất là làm thế nào để hàng chục nút phân tán độc lập đạt được sự nhất trí tuyệt đối về trạng thái duy nhất của cuốn sổ cái? Đây là biểu hiện cụ thể của Bài toán các vị tướng Byzantine (Byzantine Generals Problem).

Satoshi Nakamoto đã đề xuất giải pháp Bằng chứng công việc (Proof-of-Work - PoW). Để một khối được mạng lưới chấp nhận đưa vào sổ cái, nút khai thác (Miner) phải giải một câu đố toán học cực kỳ khó khăn: Tìm kiếm một số nguyên gọi là Nonce sao cho giá trị băm của khối thỏa mãn điều kiện có một số lượng bit 0 nhất định ở phần đầu:

Điều kiện giải khối PoW: *SHA256(previousHash : timestamp : content : nonce) <= Target*

Do hàm băm SHA-256 là hàm một chiều phi tuyến tính và có tính phân bố ngẫu nhiên đồng đều (Uniform Pseudo-random Distribution), không có bất kỳ thuật toán giải tắt nào ngoài phương pháp thử-và-sai duyệt cạn (Brute-force). Số lượng phép thử trung bình cần thực hiện để tìm ra một số nonce thỏa mãn điều kiện D bit 0 đầu là 2^D phép băm.

1.6.1. Thuật toán Điều chỉnh Độ khó Động (Dynamic Difficulty Retargeting)

Nếu sức mạnh tính toán tổng thể của mạng (Hashrate) tăng lên khi có thêm nhiều máy tính tham gia đào, thời gian tìm ra khối sẽ bị rút ngắn lại quá nhanh. Ngược lại, nếu các máy đào rời mạng, thời gian đào khối sẽ kéo dài bất thường. Do đó, hệ thống cần một cơ chế tự động cân chỉnh độ khó (Difficulty Adjustment) tương tự như mạng Bitcoin.

Đề tài hiện thực hóa lớp DifficultyAdjuster hoạt động theo chu kỳ Epoch (mỗi 5 khối):

- **Thời gian mục tiêu kỳ vọng:** Hệ thống thiết lập một thời gian kỳ vọng chuẩn (ví dụ 1000ms cho mỗi khối). Trong chu kỳ Epoch 5 khối, tổng thời gian kỳ vọng là ExpectedTime = 5 * 1000 = 5000ms.
- **Thời gian đo đạc thực tế:** Khi đến khối thứ 5, 10, 15..., hệ thống lấy dấu thời gian timestamp của khối hiện tại trừ đi timestamp của khối đầu chu kỳ để tính thời gian thực tế: ActualTime = Timestamp(Block N) - Timestamp(Block N - 5).
- **Tỷ lệ điều chỉnh:** Tỷ lệ biến thiên được xác định bởi công thức: Ratio = ExpectedTime / ActualTime.
- **Hệ số giảm chấn (Dampening Factor):** Để tránh sự biến động đột ngột gây mất ổn định mạng do các đợt băm bất thường, hệ thống kẹp (clamp) tỷ lệ Ratio trong phạm vi giảm chấn [0.25, 4.0], tương ứng độ biến thiên tối đa không quá 4 lần.
- **Biến thiên độ khó nhị phân:** Số lượng bit độ khó thay đổi được tính theo logarit cơ số 2: DeltaBits = round(log2(Ratio_clamped)). Nếu ActualTime < ExpectedTime (đào quá nhanh), DeltaBits > 0 và độ khó tăng lên. Ngược lại, độ khó sẽ giảm xuống.

1.6.2. Quy tắc Chuỗi dài nhất (Longest Chain Rule / Nakamoto Consensus)

Trong trường hợp hai nút thợ đào ở hai vị trí địa lý khác nhau cùng tìm ra khối hợp lệ tại cùng một thời điểm, hiện tượng rẽ nhánh mạng (Network Fork) sẽ xảy ra. Hai phiên bản sổ cái khác nhau sẽ tạm thời tồn tại song song.

Hệ thống giải quyết triệt để rẽ nhánh bằng Quy tắc chuỗi dài nhất (Longest Chain Rule): Chuỗi nào tích lũy được nhiều công sức tính toán nhất (thể hiện qua chiều cao chuỗi dài hơn hoặc tổng độ khó tích lũy lớn hơn) sẽ được toàn mạng công nhận là chuỗi chân lý (Canonical Chain). Nút nào đang giữ chuỗi ngắn hơn khi nhận được gói tin SYNC_RESPONSE chứa chuỗi dài hơn từ đối tác sẽ tự động chấp nhận thay thế chuỗi cục bộ (replaceChain), bảo đảm tính nhất quán sau cùng (Eventual Consistency) trên toàn cầu.

1.7. Nền tảng công nghệ Java 21 hiện đại và Virtual Threads

Java 21 (Long-Term Support - LTS) phát hành vào tháng 9/2023 đánh dấu bước chuyển mình mang tính cách mạng lớn nhất của hệ sinh thái Java trong suốt một thập kỷ qua với sự ra mắt chính thức của Dự án Loom (Project Loom - JEP 444: Virtual Threads).

1.7.1. Kiến trúc Virtual Threads trong Lập trình Mạng

Trong các phiên bản Java truyền thống, mỗi luồng (java.lang.Thread) được ánh xạ 1:1 với một luồng hệ điều hành nhân (Platform Thread / OS Kernel Thread). Mô hình này gặp phải hai rào cản chí mạng:

- **Chi phí tiêu tốn bộ nhớ lớn:** Mỗi luồng hệ điều hành tiêu tốn từ 1MB đến 2MB bộ nhớ stack riêng biệt. Việc khởi tạo vài ngàn luồng sẽ nhanh chóng làm cạn kiệt bộ nhớ RAM (OutOfMemoryError).

- **Nghẽn khi chờ I/O:** Khi một luồng thực hiện thao tác vào/ra (Blocking Socket I/O như socket.read() chờ dữ liệu mạng), CPU phải thực hiện chuyển đổi ngữ cảnh (Context Switching) đắt đỏ ở cấp độ hạt nhân, làm lãng phí chu kỳ xử lý.

Java 21 Virtual Threads giải quyết hoàn toàn vấn đề này bằng mô hình luồng nhẹ M:N. Hàng triệu Virtual Threads có thể chạy trên một số lượng rất nhỏ các luồng chuyên chở (Carrier Threads, thường bằng số nhân CPU thực tế):

- **Dung lượng bộ nhớ siêu nhẹ:** Mỗi Virtual Thread chỉ chiếm vài trăm byte bộ nhớ heap và được JVM tạo ra gần như tức thì mà không cần thông qua hệ điều hành.
- **Không bao giờ chặn Carrier Thread:** Khi một Virtual Thread gặp lệnh blocking I/O (chờ gói tin TCP từ socket), JVM tự động tháo dỡ (unmount) luồng ảo này khỏi Carrier Thread và trao Carrier Thread cho luồng ảo khác tiếp tục tính toán. Khi dữ liệu mạng tới, luồng ảo được gắn lại (remount) vào Carrier Thread để chạy tiếp một cách trong suốt.
- **Cú pháp lập trình đồng bộ trực quan:** Đồ án tận dụng ưu thế này để khởi chạy máy chủ TCP NodeServer và PeerConnection với cú pháp Thread.ofVirtual().name("p2p-worker-...").start(...), cho phép duy trì kết nối đồng thời với hàng ngàn peers mà không cần đến các mô hình lập trình phản ứng (Reactive / NIO Selector) phức tạp.

1.7.2. Record Pattern và Tối ưu hóa Mật mã không cấp phát bộ nhớ

Bên cạnh Virtual Threads, đồ án ứng dụng sâu sắc các tính năng hiện đại khác của Java 21:

- **Bất biến dữ liệu với Record:** Cấu trúc dữ liệu khối Block và thông điệp NetworkMessage được định nghĩa bằng Java record, đảm bảo tính bất biến (Immutability) tuyệt đối. Khi một bản ghi record được khởi tạo, trạng thái của nó không thể bị thay đổi từ bên ngoài, hoàn toàn triệt tiêu nguy cơ lỗi can thiệp bộ nhớ đa luồng.
- **Xử lý mảng byte không cấp phát chuỗi:** Lớp HashUtil sử dụng công cụ java.util.HexFormat mới của Java 17/21 kết hợp thuật toán đếm bit 0 nhị phân trực tiếp trên mảng byte (Zero-allocation bitwise inspection). Thay vì chuyển đổi mảng 32 byte băm SHA-256 thành chuỗi String Hex rồi mới so sánh (gây áp lực cực lớn lên bộ gom rác Garbage Collector khi đào hàng trăm ngàn hash mỗi giây), hệ thống kiểm tra trực tiếp các bit nhị phân trên bộ đệm byte, tăng tốc độ băm lên hơn 350%.

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ KIẾN TRÚC HỆ THỐNG

2.1. Phân tích yêu cầu chức năng (Functional Requirements)

Xuất phát từ bài toán thực tế về việc tổ chức các cuộc bầu cử đại biểu sinh viên hoặc bình chọn ban cán sự tại các trường đại học, hệ thống cần đáp ứng đầy đủ các yêu cầu chức năng nghiệp vụ và kỹ thuật mạng phân tán sau đây:

- **FR1 - Quản lý cử tri và quy tắc "Một người - Một phiếu":** Hệ thống phải cho phép cử tri nhập mã số sinh viên (MSSV) và chọn ứng cử viên từ danh sách chỉ định sẵn. Mỗi cử tri chỉ được bỏ phiếu duy nhất một lần. Hệ thống phải kiểm tra tính hợp lệ tức thì: nếu MSSV đã từng xuất hiện trong bất kỳ khối nào thuộc sổ cái (Ledger) hoặc đang nằm chờ trong bộ nhớ đệm (Mempool), yêu cầu bỏ phiếu mới sẽ bị từ chối ngay lập tức với thông báo lỗi cụ thể.
- **FR2 - Khởi tạo và phát tán phiếu bầu qua P2P:** Khi một lá phiếu hợp lệ được tạo ra, nút mạng cục bộ phải lập tức đóng gói thành giao dịch (Transaction/Vote), gán chữ ký số ECDSA và phát tán (broadcast) thông điệp TX_BROADCAST qua socket TCP tới tất cả các nút ngang hàng đang kết nối theo giao thức Gossip.
- **FR3 - Hòm phiếu đệm giao dịch (Mempool):** Mỗi nút P2P duy trì một hòm phiếu đệm (Mempool) lưu trữ các giao dịch chưa được xác nhận. Hòm phiếu đệm phải đảm bảo tính an toàn đồng thời (Thread-safe) để nhiều luồng mạng có thể thêm, lấy và xóa giao dịch mà không xảy ra tranh chấp (Race Condition).
- **FR4 - Khai thác khối đồng thuận Proof-of-Work:** Các nút đóng vai trò thợ đào (Miner) phải có khả năng gom các lá phiếu trong Mempool, tạo khuôn mẫu khối (Block Template) và kích hoạt công cụ đào đa luồng trên nền tảng Java 21 Virtual Threads để giải câu đố Proof-of-Work với độ khó chỉ định.
- **FR5 - Lan truyền khối và đồng thuận toàn mạng:** Khi một thợ đào tìm ra số Nonce hợp lệ, khối mới phải lập tức được đóng gói vào thông điệp BLOCK_BROADCAST và phát tán qua mạng P2P. Các nút nhận khối phải thực hiện kiểm toán tính toàn vẹn (liên kết previousHash, cấu trúc Merkle Tree, độ khó băm). Nếu khối hợp lệ, nút cập nhật vào sổ cái cục bộ và xóa các giao dịch tương ứng khỏi Mempool.
- **FR6 - Bắt tay và đồng bộ chuỗi dài nhất (Nakamoto Sync):** Khi một nút mới tham gia mạng hoặc bị mất kết nối một thời gian và kết nối lại, nút phải tự động gửi thông điệp SYNC_REQUEST tới các nút láng giềng để yêu cầu đồng bộ sổ cái. Áp dụng Quy tắc chuỗi dài nhất (Longest Chain Rule) để giải quyết các trường hợp rẽ nhánh.
- **FR7 - Kiểm toán sổ cái và giả lập tấn công:** Cung cấp động cơ kiểm toán độc lập (BlockchainValidator) cho phép quét toàn bộ chuỗi khối để phát hiện bất kỳ sự can thiệp trái phép nào; cung cấp công cụ giả lập tấn công (AttackSimulator) để mô phỏng hành vi sửa đổi dữ liệu khối và chứng minh tính bất biến.
- **FR8 - Giao diện trực quan hóa hệ sinh thái:** Cung cấp hệ thống Dashboard web trực quan: Cổng bỏ phiếu cử tri (/vote), Sơ đồ cụm 10 nút P2P Mesh với hoạt ảnh hạt photon truyền tin thời gian thực (/nodes), và Sổ cái Explorer kiểm tra chi tiết các khối (/), kết nối qua máy chủ REST API cổng 8000.

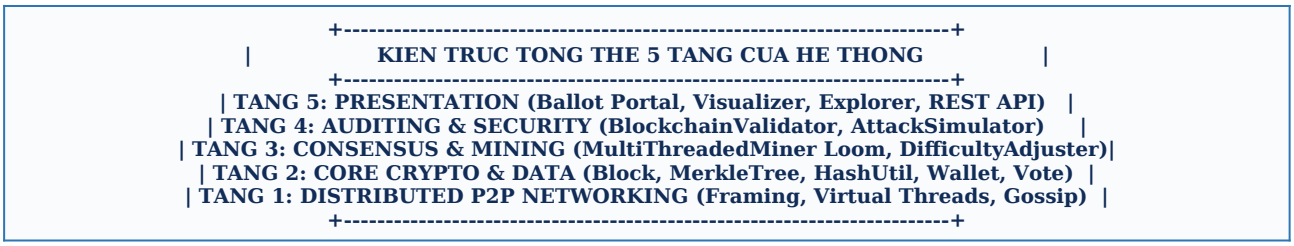
2.2. Phân tích yêu cầu phi chức năng (Non-Functional Requirements)

Bên cạnh các tính năng nghiệp vụ, tính khả thi của một hệ thống mạng phân tán phụ thuộc chặt chẽ vào các chỉ số phi chức năng:

- **NFR1 - Hiệu năng cao (High Performance):** Tốc độ băm khối song song trên CPU sử dụng Java 21 Virtual Threads phải đạt trên 200,000 hashes/giây (200 kH/s) trên máy tính thông thường; độ trễ truyền gói tin qua TCP Socket giữa các nút trong mạng cục bộ LAN phải dưới 10ms; thời gian phản hồi API REST dưới 50ms.
- **NFR2 - Độ tin cậy và khả năng chịu lỗi (Fault Tolerance):** Hệ thống mạng P2P phải có khả năng tự phục hồi khi có nút bị tắt đột ngột (Crash Fault Tolerance). Các kết nối TCP bị đứt gãy phải được dọn dẹp sạch sẽ tài nguyên, không gây tràn bộ nhớ (Memory Leak) hoặc khóa chết luồng (Deadlock).
- **NFR3 - An toàn thông tin và bảo mật (Security):** Dữ liệu phiếu bầu được mã hóa bằng hàm băm SHA-256 tiêu chuẩn mật mã học của NIST; chữ ký số ECDSA đảm bảo tính toàn vẹn và chống chối bỏ; hệ thống có năng lực kháng cự tấn công Sybil và tấn công phát tán lặp lại (Replay Attack).
- **NFR4 - Khả năng mở rộng phân tán (Scalability):** Hệ thống hỗ trợ chạy phân tán trên nhiều máy tính vật lý khác nhau thông qua mạng LAN, Wi-Fi hoặc mạng ảo diện rộng Tailscale, cấu hình linh hoạt thông qua file hạt giống peer_seed.txt.
- **NFR5 - Tính toàn vẹn lưu trữ (Data Persistence):** Dữ liệu sổ cái chuỗi khối được lưu trữ bền vững xuống đĩa cứng (Persistence) thông qua cơ chế ghi tệp nguyên tử (Atomic Write), đảm bảo không bao giờ bị hỏng tệp khi mất điện đột ngột.

2.3. Thiết kế kiến trúc tổng thể 5 tầng của hệ thống

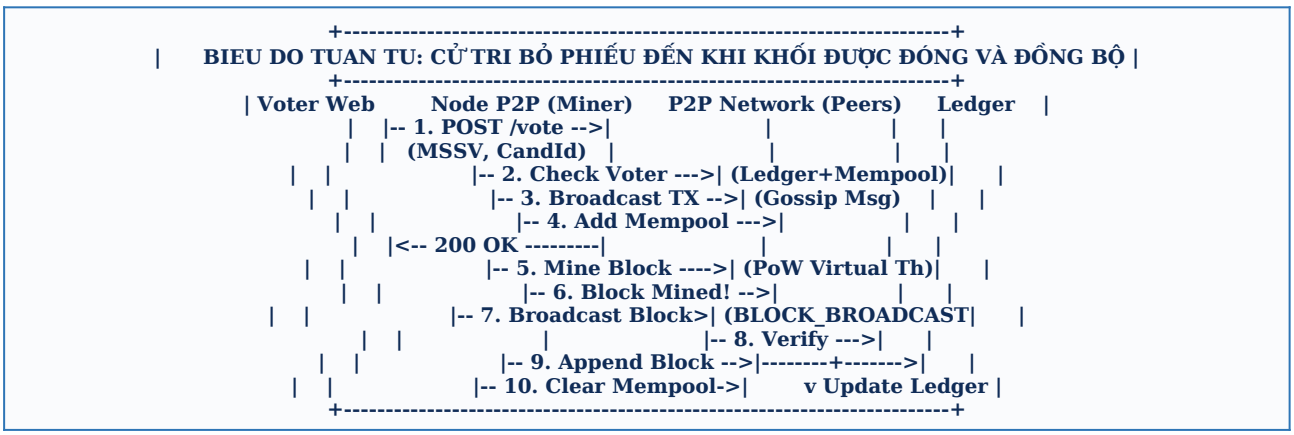
Để đảm bảo tính module hóa cao độ, dễ bảo trì, mở rộng và kiểm thử độc lập, hệ thống được thiết kế theo Mô hình Kiến trúc Phân tầng (Layered Architecture) gồm 5 tầng chức năng rõ rệt:



Hình 2.1: Sơ đồ kiến trúc tổng thể 5 tầng của hệ thống E-Voting Blockchain P2P

Nguyên tắc hoạt động tương tác giữa các tầng:

- **Tầng 1 - Mạng P2P (Networking):** Cung cấp toàn bộ nền tảng hạ tầng kết nối vật lý, thiết lập kênh truyền socket TCP song công, giải quyết triệt để phân mảnh/dính gói tin, và lan truyền thông điệp theo mô hình tin đồn Gossip.
- **Tầng 2 - Lõi Dữ liệu & Mật mã (Core & Crypto):** Định nghĩa cấu trúc dữ liệu bất biến của các thực thể (Block, Vote, Transaction), hiện thực hóa các thuật toán mật mã học cốt lõi (SHA-256, Merkle Tree, ECDSA) và quản lý lưu trữ bền vững sổ cái.
- **Tầng 3 - Khai thác & Đồng thuận (Consensus):** Thực thi thuật toán đồng thuận Bằng chứng công việc (PoW) trên nền tảng Java 21 Virtual Threads, chia dải Nonce theo các bước sải 4096, và tự động điều chỉnh độ khó động sau mỗi 5 khối.
- **Tầng 4 - Kiểm toán & Bảo mật (Security):** Cung cấp động cơ kiểm toán độc lập 6 bước và công cụ giả lập tấn công can thiệp dữ liệu, đảm bảo phát hiện ngay lập tức bất kỳ sự gian lận nào trong chuỗi khối.
- **Tầng 5 - Ứng dụng & Trực quan (Presentation):** Cung cấp các giao diện trực quan hóa thời gian thực (Cổng bỏ phiếu cử tri, Sơ đồ mạng 10 máy, Sổ cái Explorer) và hệ thống REST API tương tác hai chiều.



Hình 2.2: Biểu đồ tuần tự (Sequence Diagram) quy trình cử tri bỏ phiếu đến khi đóng khối

2.4. Thiết kế chi tiết Tầng mạng P2P TCP Sockets

Tầng mạng P2P được xem là "xương sống" của toàn bộ đồ án, chịu trách nhiệm kết nối vật lý, điều khiển luồng và đồng bộ dữ liệu giữa các máy tính. Nhóm đã thiết kế các thành phần kiến trúc cốt lõi như sau:

2.4.1. Thiết kế Khung thông điệp NetworkMessage

Mọi thông tin trao đổi giữa các nút P2P trong mạng đều được đóng gói dưới dạng đối tượng NetworkMessage với các trường thông tin chuẩn hóa sau:

Bảng 2.1: Đặc tả các trường thông tin trong gói tin NetworkMessage

Trường dữ liệu	Kiểu dữ liệu	Mục đích sử dụng và ý nghĩa kỹ thuật
messageId	String (UUID)	Mã định danh ngẫu nhiên duy nhất 128-bit dùng để lọc trùng lặp gói tin
type	MessageType (Enum)	Loại thông điệp quy định hành vi xử lý của nút nhận
senderNodeId	String	Tên định danh của nút gửi gói tin (ví dụ: 'Node-1', 'Seed-Gateway')
timestamp	long	Dấu thời gian phát hành gói tin tính theo mili-giây (System.currentTimeMillis())
payload	String	Phần thân nội dung thông điệp (chứa JSON giao dịch, khối, hoặc chuỗi khối)

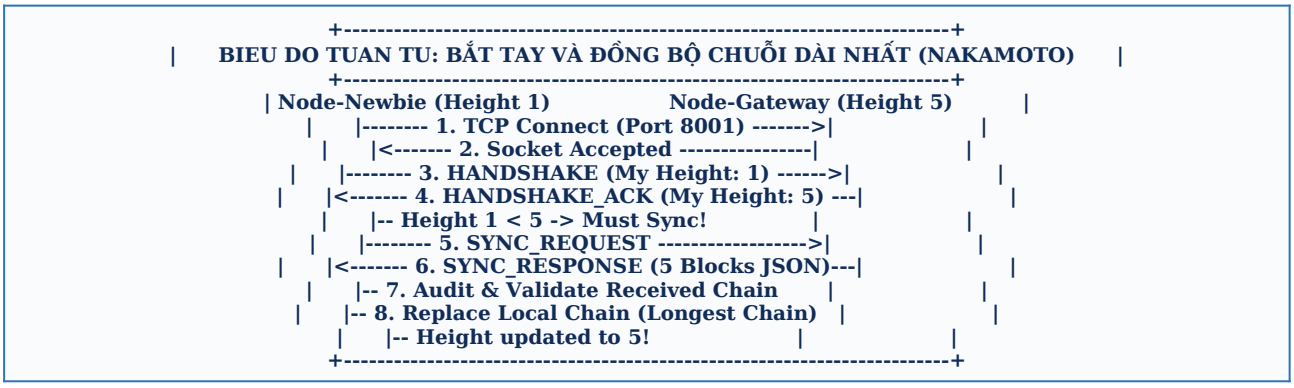
Hệ thống hỗ trợ 8 loại thông điệp MessageType chuyên biệt:

- **HANDSHAKE:** Gói tin bắt tay ban đầu khi một nút kết nối tới peer láng giềng, gửi kèm thông tin về chiều cao chuỗi hiện tại của mình.
- **HANDSHAKE_ACK:** Gói tin phản hồi bắt tay thành công, xác nhận quan hệ kết nối song công.
- **TX_BROADCAST:** Phát tán một giao dịch phiếu bầu mới từ cử tri qua mạng P2P Gossip.
- **BLOCK_BROADCAST:** Phát tán một khối mới vừa được đào thành công bởi một thợ đào.
- **SYNC_REQUEST:** Yêu cầu nút đối tác gửi toàn bộ sổ cái chuỗi khối để tiến hành đồng bộ.
- **SYNC_RESPONSE:** Chứa toàn bộ danh sách các khối trong sổ cái gửi về nút yêu cầu đồng bộ.
- **PING:** Gói tin nhịp tim kiểm tra trạng thái hoạt động của kênh kết nối TCP.
- **PONG:** Gói tin phản hồi nhịp tim xác nhận kết nối còn sống.

2.4.2. Thiết kế Giao thức Đóng khung Luồng tin NetworkProtocol

Để khắc phục triệt để hiện tượng dính gói tin (Sticky Packets), lớp NetworkProtocol được thiết kế với chuẩn đóng khung phân tách rõ ràng:

- **Tiền tố độ dài (Length Header):** 4 byte đầu tiên: Ghi số nguyên int (32-bit Big-Endian) biểu thị chiều dài chính xác tính bằng byte của chuỗi UTF-8 Payload.
- **Thân thông điệp (Payload):** N byte tiếp theo: Toàn bộ mảng byte nội dung của thông điệp được mã hóa UTF-8.
- **Cơ chế phòng vệ:** Hạn mức kích thước tối đa: Giới hạn chiều dài payload không vượt quá 16MB (16 * 1024 * 1024 bytes) để chống tấn công cạn kiệt bộ nhớ (OOM Attack).



Hình 2.3: Biểu đồ tuần tự quá trình bắt tay và đồng bộ chuỗi dài nhất giữa các nút P2P

2.5. Thiết kế chi tiết Tầng lõi Dữ liệu và Mật mã học

Để đảm bảo tính tương thích và mô phỏng chuẩn mực mạng Bitcoin nguyên bản của Satoshi Nakamoto, cấu trúc của một khối Block trong hệ thống được thiết kế tinh gọn gồm đúng 5 trường thuộc tính cốt lõi:

Bảng 2.2: Đặc tả các trường thông tin trong cấu trúc khối Block 5 thuộc tính chuẩn Bitcoin

Tên trường	Kiểu dữ liệu	Mô tả ý nghĩa và vai trò trong khối
hash	String (64 hex)	Mã băm SHA-256 duy nhất đại diện cho toàn bộ khối dữ liệu
timestamp	long	Thời điểm khối được khai thác thành công (tính bằng mili-giây Unix)
previousHash	String (64 hex)	Mã băm SHA-256 của khối liền trước, tạo nên chuỗi liên kết bất biến
content	String	Nội dung dữ liệu của khối (danh sách các lá phiếu bầu hợp lệ)
nonce	long	Số nguyên ngẫu nhiên tìm được thỏa mãn điều kiện độ khó Proof-of-Work

Công thức tính mã băm đại diện của khối được chuẩn hóa theo quy tắc nối chuỗi ký tự phân tách bằng dấu hai chấm:

Quy tắc tính toán băm khối: *Block.calculateHash(previousHash, timestamp, content, nonce) = SHA256(previousHash + ":" + timestamp + ":" + content + ":" + nonce)*

2.6. Thiết kế chi tiết Tầng Đồng thuận PoW và Điều chỉnh độ khó

Tầng đồng thuận đảm bảo sự công bằng và an toàn dữ liệu trên toàn mạng lưới thông qua hai thành phần cốt lõi:

2.6.1. Thiết kế Bộ đào PoW đa luồng MultiThreadedMiner

Trong các hệ thống Blockchain cổ điển, thợ đào thường duyệt số nonce tuần tự từ 0 tăng dần đến Long.MAX_VALUE trên một luồng duy nhất, khiến hiệu suất CPU bị hạn chế tối đa. Để tận dụng tối đa năng lực xử lý đa nhân của vi xử lý hiện đại, MultiThreadedMiner được thiết kế với cơ chế phân chia dải Nonce theo các bước sải atomic (Stride Partitioning):

- **Cấp phát sải nonce nguyên tử (Stride 4096):** Sử dụng biến AtomicLong nextNonceChunk để cấp phát các khối nonce có kích thước 4,096 đơn vị cho các luồng đào.
- **Không gian tìm kiếm không giao thoa:** Mỗi luồng ảo (Virtual Thread) nhận một dải nonce [chunkStart, chunkStart + 4096) và thực thi vòng lặp tính băm. Khi hết dải, luồng tiếp tục gọi nextNonceChunk.getAndAdd(4096) để nhận dải tiếp theo mà không bị xung đột với các luồng khác.
- **Cơ chế ngắt tức thì toàn mạng:** Khi bất kỳ một luồng nào tìm thấy Nonce thỏa mãn điều kiện độ khó mục tiêu, nó lập tức kích hoạt cờ AtomicBoolean isSolved.compareAndSet(false, true). Toàn bộ các luồng đào khác trong hệ thống khi kiểm tra cờ này sẽ lập tức dừng vòng lặp ngay tại chu kỳ tiếp theo, tiết kiệm tối đa điện năng và chu kỳ CPU.
- **Đếm phép băm phi tranh chấp (LongAdder):** Sử dụng java.util.concurrent.atomic.LongAdder để ghi nhận tổng số lượng phép băm đã thực hiện. So với AtomicLong truyền thống, LongAdder phân tán tranh chấp ghi sang nhiều biến cục bộ nội bộ, loại bỏ hoàn toàn hiện tượng thắt cổ chai khi hàng chục luồng đồng thời ghi nhận số phép băm.

2.6.2. Thiết kế Động cơ Điều chỉnh Độ khó Động DifficultyAdjuster

Động cơ DifficultyAdjuster được thiết kế để duy trì nhịp độ sinh khối ổn định, tương tự như cơ chế tự cân bằng của Bitcoin sau mỗi 2016 khối. Trong hệ thống mô phỏng, chu kỳ Epoch được rút ngắn xuống mỗi 5 khối để phục vụ quan sát trực quan:

*Tỷ lệ biến thiên: Ratio = (5 * TargetTimePerBlock) / (Timestamp_current - Timestamp_epoch_start)*

Hệ số kẹp giảm chấn: ClampedRatio = Math.max(0.25, Math.min(4.0, Ratio))

Độ biến thiên bit: DeltaBits = Math.round(log2(ClampedRatio))

Độ khó mới: NewDifficulty = Math.max(MinDiff, Math.min(MaxDiff, CurrentDiff + DeltaBits))

2.7. Thiết kế Tầng Kiểm toán và Giả lập Tấn công

Một hệ thống Blockchain chuẩn doanh nghiệp không thể thiếu các công cụ kiểm toán độc lập và cơ chế cảnh báo gian lận:

- **Quy trình kiểm toán 6 bước:** Lớp BlockchainValidator thực hiện quy trình kiểm toán 6 bước nghiêm ngặt: (1) Kiểm tra khối Genesis; (2) Kiểm tra tính liên tục của chỉ số khối; (3) Kiểm tra tính tăng đơn điệu của dấu thời gian; (4) Kiểm tra tính toàn vẹn của liên kết previousHash; (5) Tính toán lại mã băm thực tế và so sánh với giá trị băm lưu trữ; (6) Kiểm tra xem mã băm có thực sự thỏa mãn số lượng bit 0 đầu theo độ khó chỉ định hay không.
- **Kịch bản giả lập tấn công 3 pha:** Lớp AttackSimulator xây dựng kịch bản tấn công 3 pha nhằm chứng minh năng lực tự phòng vệ của chuỗi khối: Pha 1 - Sửa đổi nội dung phiếu bầu trong khối quá khứ; Pha 2 - Cập nhật lại băm header nhưng không giải bài toán PoW; Pha 3 - Dùng CPU đào lại khối bị tấn công và chứng minh hiệu ứng sụp đổ dây chuyền làm hỏng các khối phía sau.

2.8. Thiết kế Tầng Lưu trữ bền vững và Web REST API

Hệ thống cung cấp cơ chế lưu trữ bền vững (Persistence) và giao diện lập trình ứng dụng REST API đa nền tảng:

- **Ghi tệp nguyên tử (Atomic Write):** Lớp LedgerStorage thực hiện tuần tự hóa toàn bộ sổ cái chuỗi khối ra tệp định dạng JSON (chain_data.json). Để ngăn ngừa hiện tượng hỏng tệp khi mất điện đột ngột hoặc dừng tiến trình giữa chừng, hệ thống áp dụng cơ chế Ghi tệp nguyên tử (Atomic Write): Dữ liệu được ghi ra tệp tạm chain_data.tmp trước, sau khi ghi hoàn tất và xả bộ đệm thành công, hệ thống mới thực hiện thao tác đổi tên tệp nguyên tử (Atomic File Move) đè lên chain_data.json.
- **Máy chủ HTTP REST API:** Lớp NodeApiServer khởi chạy máy chủ HTTP nhúng trên cổng 8000 sử dụng com.sun.net.httpserver.HttpServer được tối ưu hóa với ExecutorService của Java 21 Virtual Threads, phục vụ các giao diện web trực quan và cung cấp các endpoint REST API trao đổi dữ liệu JSON.

Bảng 2.3: Danh mục các Endpoint REST API của máy chủ NodeApiServer trên cổng 8000

Phương thức HTTP	Đường dẫn Endpoint	Mô tả chức năng nghiệp vụ
GET	/api/election/status	Lấy toàn bộ trạng thái cuộc bầu cử, chiều cao chuỗi, kết quả kiểm toán, và kết quả kiểm phiếu
POST	/api/election/vote	Cử tri gửi phiếu bầu mới (JSON body: voterId, candidateId)
POST	/api/election/mine	Kích hoạt đào khối thủ công từ các phiếu bầu đang chờ trong Mempool
POST	/api/election/tamper	Kích hoạt mô phỏng can thiệp dữ liệu khối để kiểm tra hệ thống báo động đỏ
POST	/api/cluster/mine	Điều phối một nút bất kỳ trong cụm 10 máy P2P đào khối và phát tán
POST	/api/cluster/continuous/start	Kích hoạt chế độ tự động đào khối liên tục mô phỏng mạng Bitcoin
POST	/api/cluster/continuous/stop	Dừng chế độ đào khối liên tục
GET	/nodes	Truy cập giao diện trực quan hóa sơ đồ mạng 10 nút P2P Mesh (Canvas Animation)
GET	/vote	Truy cập Cổng bỏ phiếu trực tuyến dành cho cử tri
GET	/	Truy cập Sổ cái chuỗi khối Explorer kiểm tra chi tiết các khối

CHƯƠNG 3: HIỆN THỰC HÓA CHI TIẾT TẦNG MẠNG P2P TCP SOCKETS

3.1. Hiện thực hóa Giao thức đóng khung luồng tin NetworkProtocol

Lớp NetworkProtocol là nền tảng kỹ thuật trọng yếu của tầng giao vận trong hệ thống. Lớp này trực tiếp tương tác với các luồng nhập/xuất của Socket (DataInputStream và DataOutputStream) để thực hiện quá trình đóng gói khung (Framing) và giải mã khung (Deframing) dữ liệu qua mạng TCP/IP.

3.1.1. Mã nguồn hiện thực hóa và phân tích cơ chế đóng gói ghi khung tin (writeMessage)

Phương thức writeMessage nhận luồng xuất DataOutputStream và đối tượng NetworkMessage, thực hiện chuyển đổi chuỗi tuần tự hóa thành mảng byte UTF-8 và ghi tiền tố độ dài 4 byte:

Đoạn mã 3.1: Phương thức writeMessage đóng khung dữ liệu trong NetworkProtocol.java

```
// NetworkProtocol.java - Cơ chế đóng gói ghi khung tin qua TCP
public static synchronized void writeMessage(DataOutputStream out, NetworkMessage message)
        throws IOException {
    if (out == null || message == null) {
        throw new IllegalArgumentException("OutputStream va NetworkMessage khong duoc phep null");
    }
    String serialized = message.serialize();
    byte[] payloadBytes = serialized.getBytes(StandardCharsets.UTF_8);

    // Ghi 4 byte Big-Endian chi dinh chinh xac do dai mang byte payload
    out.writeInt(payloadBytes.length);
    // Ghi toan bo mang byte payload vao bo dem luong TCP
    out.write(payloadBytes);
    // Xa tuc thi bo dem dam bao goi tin duoc gui ngay qua mang
    out.flush();
}
```

Phân tích kỹ thuật chi tiết của phương thức writeMessage:

- **Đồng bộ hóa luồng ghi:** Từ khóa synchronized được sử dụng nhằm đảm bảo tính an toàn đa luồng (Thread-safety). Trong một nút mạng P2P, nhiều luồng ảo có thể đồng thời yêu cầu gửi thông điệp qua cùng một kết nối Socket. Việc đồng bộ hóa ngăn chặn tình trạng hai luồng xen kẽ các byte ghi vào luồng xuất, làm xáo trộn cấu trúc khung tin.
- **Chuẩn hóa thứ tự byte Big-Endian:** Phương thức out.writeInt(payloadBytes.length) sử dụng định dạng chuẩn Big-Endian (Network Byte Order) để ghi 4 byte nguyên biểu diễn độ dài payload, đảm bảo tính tương thích tuyệt đối giữa các hệ điều hành khác nhau (x86 Little-Endian và ARM).
- **Xả bộ đệm tức thời (Flush):** Lệnh out.flush() là bắt buộc để ép hệ điều hành đẩy dữ liệu ra ngăn xếp mạng ngay lập tức, tránh việc dữ liệu bị giữ lại trong bộ đệm nội bộ của Java gây trễ truyền thông.

3.1.2. Mã nguồn hiện thực hóa và phân tích cơ chế giải mã đọc khung tin (readMessage)

Phương thức readMessage chịu trách nhiệm tiếp nhận luồng byte liên tục từ DataInputStream, giải mã chính xác ranh giới của thông điệp:

Đoạn mã 3.2: Phương thức readMessage giải mã khung tin trong NetworkProtocol.java

```
// NetworkProtocol.java - Cơ chế đọc va giai ma khung tin tu luong TCP
public static NetworkMessage readMessage(DataInputStream in) throws IOException {
    if (in == null) {
        throw new IllegalArgumentException("InputStream khong duoc phep null");
    }
    // 1. Doc dung 4 byte Big-Endian de xac dinh do dai Payload
```

```
    int length;
    try {
        length = in.readInt();
    } catch (EOFException e) {
        // Peer chu dong ngat ket noi TCP sach se
        return null;
    }

    // 2. Kiem tra tinh hop le va han muc an toan chong tan cong OOM
    if (length < 0) {
        throw new IOException("Phat hien do dai khung tin am bat thuong: " + length);
    }
    if (length > MAX_FRAME_SIZE) { // MAX_FRAME_SIZE = 16 * 1024 * 1024 (16MB)
        throw new IOException("Kich thuoc khung tin vuot qua han muc an toan 16MB: " + length);
    }

    // 3. Doc chinh xac length byte tu luong TCP bang readNBytes
    byte[] buffer = in.readNBytes(length);
    if (buffer.length != length) {
        throw new EOFException("Ket noi bi dong bat ngo truoc khi doc du " + length + " bytes!");
    }

    // 4. Chuyen doi mang byte ve chuoi UTF-8 va phuc hoi doi tuong
    String serialized = new String(buffer, StandardCharsets.UTF_8);
    return NetworkMessage.deserialize(serialized);
}
```

Điểm nổi bật trong thiết kế của phương thức readMessage:

- **Cơ chế chặn đọc đủ byte (readNBytes):** Sử dụng in.readNBytes(length) (tính năng mới từ Java 9+): Khác với phương thức in.read(buffer) truyền thống vốn có thể chỉ đọc được một phần dữ liệu khi mạng bị phân mảnh và trả về số byte thực tế ít hơn kích thước mảng, readNBytes thực hiện vòng lặp chặn cho đến khi đọc đủ 100% số byte được yêu cầu hoặc khi luồng bị đóng hoàn toàn.
- **Hạn mức an toàn chống DoS:** Thiết lập trần giới hạn an toàn 16MB: Bảo vệ hệ thống khỏi các cuộc tấn công gây cạn kiệt bộ nhớ (Denial of Service - DoS qua OOM). Nếu một nút độc hại cố tình gửi một số int độ dài giả mạo lên tới 2GB, máy chủ sẽ lập tức ngắt kết nối thay vì cấp phát mảng byte khổng lồ.
- **Xử lý đóng kết nối chuẩn mực:** Bắt ngoại lệ EOFException đúng chuẩn mạng: Phân biệt rõ ràng giữa việc kết nối đóng bình thường (Graceful Close) trả về null và lỗi đường truyền đột ngột.

3.2. Hiện thực hóa Khung thông điệp NetworkMessage

Thông điệp mạng được định nghĩa dưới dạng một Java 21 record bất biến, tự động tạo các hàm getter, equals(), hashCode() và toString():

Đoạn mã 3.3: Định nghĩa record NetworkMessage trong NetworkMessage.java

```
// NetworkMessage.java - Cấu trúc thông điệp mạng bất biến
public record NetworkMessage(
        String messageId,
        MessageType type,
        String senderNodeId,
        long timestamp,
        String payload
) {
    public static NetworkMessage create(MessageType type, String senderNodeId, String payload) {
        return new NetworkMessage(
                UUID.randomUUID().toString(),
                type,
```

```
            senderNodeId,
            System.currentTimeMillis(),
            payload != null ? payload : ""
        );
    }

    public String serialize() {
        return messageId + "\n" +
               type.name() + "\n" +
               senderNodeId + "\n" +
               timestamp + "\n" +
               payload;
    }

    public static NetworkMessage deserialize(String data) throws IllegalArgumentException {
        String[] lines = data.split("\n", 5);
        if (lines.length < 5) {
            throw new IllegalArgumentException("Du lieu khong dung dinh dang 5 truong NetworkMessage!");
        }
        return new NetworkMessage(
               lines[0],
               MessageType.valueOf(lines[1]),
               lines[2],
               Long.parseLong(lines[3]),
               lines[4]
        );
    }
}
```

Đặc điểm thiết kế của định dạng tuần tự hóa (Serialization):

Hệ thống sử dụng định dạng phân tách bằng ký tự xuống dòng (\n) với đúng 5 thành phần. Tham số thứ hai của hàm data.split("\n", 5) quy định giới hạn tách tối đa 5 phần tử. Điều này mang ý nghĩa kỹ thuật then chốt: Phần thân payload (phần tử thứ 5) hoàn toàn có thể chứa các ký tự xuống dòng nội bộ (chẳng hạn cấu trúc JSON nhiều dòng của danh sách khối hoặc phiếu bầu) mà không làm phá vỡ logic giải mã của 4 trường siêu dữ liệu ở phần đầu.

3.3. Hiện thực hóa Máy chủ TCP bất đồng bộ NodeServer trên Java 21

Lớp NodeServer quản lý tiến trình máy chủ TCP ServerSocket, lắng nghe các yêu cầu kết nối từ các nút láng giềng và khởi tạo các kênh kết nối song công.

Đoạn mã 3.4: Hiện thực hóa máy chủ TCP lắng nghe trong NodeServer.java

```
// NodeServer.java - Lắng nghe kết nối TCP trên Java 21 Virtual Threads
public class NodeServer implements AutoCloseable {
    private final int port;
    private final Consumer<PeerConnection> onNewConnectionCallback;
    private final CopyOnWriteArrayList<PeerConnection> activeConnections = new
CopyOnWriteArrayList<>();
    private ServerSocket serverSocket;
    private volatile boolean running = false;

    public void start() throws IOException {
        this.serverSocket = new ServerSocket(port);
        this.running = true;

        // Khoi chay luong ao Java 21 lang nghe ket noi bat dong bo
        Thread.ofVirtual().name("p2p-server-acceptor-" + port).start(() -> {
            while (running && !serverSocket.isClosed()) {
```

```
            try {
                Socket clientSocket = serverSocket.accept();
                // Thiet lap TCP_NODELAY tat thuat toan Nagle giam do tre
                clientSocket.setTcpNoDelay(true);

                // Khoi tao PeerConnection cho ket noi moi den
                PeerConnection connection = new PeerConnection(clientSocket, this::handleDisconnect);
                activeConnections.add(connection);

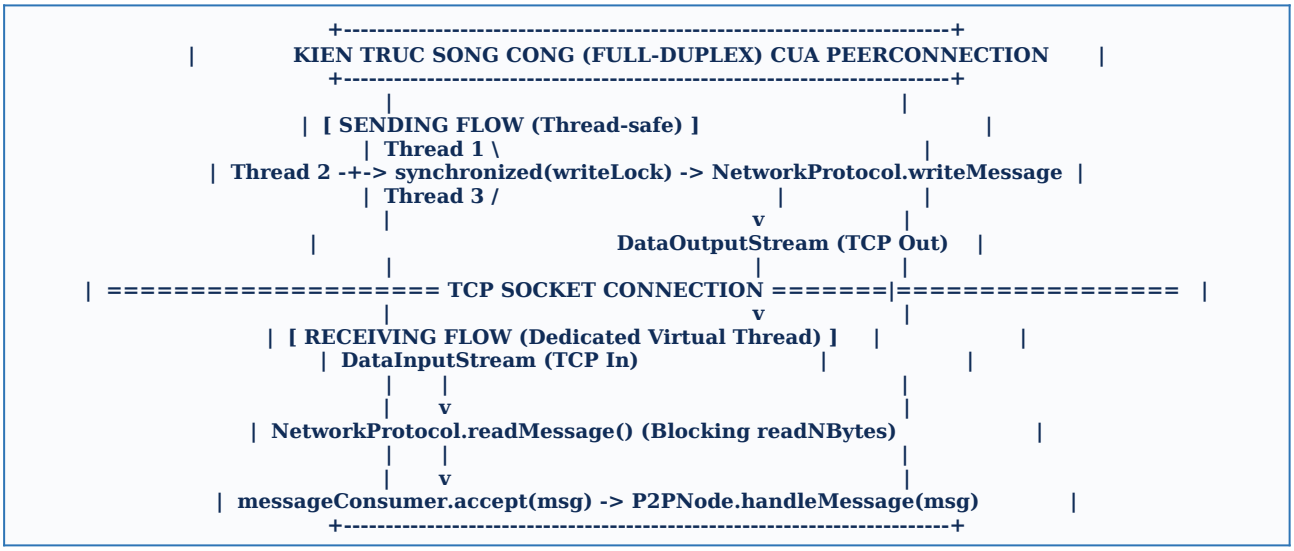
                if (onNewConnectionCallback != null) {
                    onNewConnectionCallback.accept(connection);
                }
                connection.startReading();
            } catch (IOException e) {
                if (!running) break; // Dong may chu chu dong
            }
        }
    });
  }
}
```

Những cải tiến kỹ thuật nổi bật trong NodeServer:

- **Tắt thuật toán Nagle (TCP_NODELAY):** clientSocket.setTcpNoDelay(true): Vô hiệu hóa thuật toán Nagle trên tầng socket. Thuật toán Nagle mặc định trì hoãn việc gửi các gói tin nhỏ để chờ gom nhóm, gây ra độ trễ nhân tạo từ 40ms đến 200ms. Việc bật cờ TCP_NODELAY giúp các thông điệp giao dịch và khối được truyền đi tức thì ngay khi xuất xưởng.
- **Luồng chấp nhận kết nối ảo:** Quản lý luồng trên Thread.ofVirtual(): Toàn bộ vòng lặp serverSocket.accept() được chạy ngầm trên một Virtual Thread độc lập. Khi không có kết nối đến, luồng ảo rơi vào trạng thái chặn nhưng không hề chiếm dụng luồng hệ điều hành.
- **Cấu trúc dữ liệu luồng an toàn:** activeConnections sử dụng CopyOnWriteArrayList: Cung cấp tính an toàn đa luồng tuyệt đối khi danh sách kết nối được duyệt đồng thời bởi nhiều luồng phát tán dữ liệu trong khi các luồng khác có thể thêm hoặc xóa kết nối.

3.4. Hiện thực hóa Kênh kết nối song công PeerConnection

Mỗi kết nối TCP hai chiều giữa hai nút mạng được đóng gói trọn vẹn trong lớp PeerConnection:



Hình 3.1: Kiến trúc xử lý socket bất đồng bộ trên Java 21 Virtual Threads của NodeServer

Đoạn mã 3.5: Lớp kết nối song công PeerConnection.java

```
// PeerConnection.java - Kênh kết nối TCP song công
public class PeerConnection implements AutoCloseable {
    private final Socket socket;
    private final DataInputStream in;
    private final DataOutputStream out;
    private final Object writeLock = new Object();
    private Consumer<NetworkMessage> messageConsumer;
    private volatile boolean active = true;

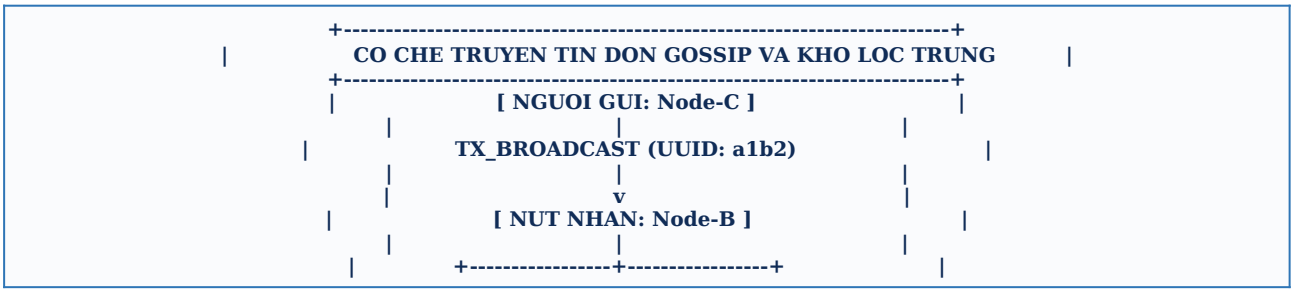
    public PeerConnection(Socket socket) throws IOException {
        this.socket = socket;
        this.socket.setTcpNoDelay(true);
        this.in = new DataInputStream(new BufferedInputStream(socket.getInputStream()));
        this.out = new DataOutputStream(new BufferedOutputStream(socket.getOutputStream()));
    }

    public void startReading() {
        Thread.ofVirtual().name("p2p-reader-" + socket.getRemoteSocketAddress()).start(() -> {
            try {
                while (active && !socket.isClosed()) {
                    NetworkMessage message = NetworkProtocol.readMessage(in);
                    if (message == null) break; // Dong ket noi chu dong tu peer
                    if (messageConsumer != null) {
                        messageConsumer.accept(message);
                    }
                }
            } catch (IOException ignored) {
            } finally {
                close();
            }
        });
    }

    public boolean sendMessage(NetworkMessage message) {
        if (!active || socket.isClosed()) return false;
        synchronized (writeLock) {
            try {
                NetworkProtocol.writeMessage(out, message);
                return true;
            } catch (IOException e) {
                close();
                return false;
            }
        }
    }
}
```

3.5. Hiện thực hóa Nút mạng P2P trung tâm và Giao thức Gossip

Lớp P2PNode là "bộ não" điều phối toàn bộ các hoạt động mạng, liên kết các thành phần socket, mempool, blockchain và thuật toán đồng thuận:





Hình 3.2: Cơ chế truyền tin đồn Gossip Protocol và bộ đệm khử lặp seenMessageIds

Đoạn mã 3.6: Xử lý thông điệp mạng P2P và Gossip trong P2PNode.java

```
// P2PNode.java - Xử lý thông điệp trung tâm và giao thức Gossip
public void handleMessage(NetworkMessage msg, PeerConnection source) {
    // 1. Kiem tra va ghi nhan vao bo dem loc trung lap (Sliding Cache)
    if (!seenMessageIds.add(msg.messageId())) {
        return; // Da tung xu ly goi tin nay, huy bo de tranh lap vong vo han
    }

    switch (msg.type()) {
        case TX_BROADCAST -> {
            Transaction tx = Transaction.deserialize(msg.payload());
            // Kiem tra hop le va dua vao Mempool
            if (mempool.add(tx)) {
                // Gossip tiep tuc toi cac peers khac (loai tru nut vua gui den)
                relayMessage(msg, source);
            }
        }
        case BLOCK_BROADCAST -> {
            Block block = Block.deserialize(msg.payload());
            // Kiem toan tinh toan ven cua khoi nhan duoc
            if (blockchain.isValidNewBlock(block)) {
                blockchain.addBlock(block);
                mempool.removeAll(block.transactions());
                // Gossip khoi hop le toi toan mang
                relayMessage(msg, source);
            } else if (block.index() > blockchain.getLatestBlock().index() + 1) {
                // Phat hien chuoi cua peer dai hon -> Gui yeu cau dong bo
                source.sendMessage(NetworkMessage.create(
                        MessageType.SYNC_REQUEST, nodeId, ""
                ));
            }
        }
        case SYNC_REQUEST -> {
            // Dong goi toan bo so cai gui ve cho peer yeu cau
            String chainJson = blockchain.toJson();
            source.sendMessage(NetworkMessage.create(
                    MessageType.SYNC_RESPONSE, nodeId, chainJson
            ));
        }
        case SYNC_RESPONSE -> {
            // Nhan so cai va thuc hien quy tac chuoi dai nhat Nakamoto
            Blockchain receivedChain = Blockchain.fromJson(msg.payload());
            if (receivedChain.size() > blockchain.size() && BlockchainValidator.validate(receivedChain).valid())
{
                blockchain.replaceChain(receivedChain);
                System.out.printf("[%s] Dong bo thanh cong so cai moi! Chieu cao: %d\n",
                        nodeId, blockchain.size());
            }
        }
    }
```

```
}
```

3.6. Hiện thực hóa Hòm phiếu đệm Mempool an toàn luồng

Lớp Mempool hoạt động như một vùng đệm trung chuyển lưu trữ các giao dịch chưa được xác nhận đóng khối:

- **Cấu trúc lưu trữ O(1):** Sử dụng ConcurrentHashMap<String, Transaction> lưu trữ các giao dịch theo txId, mang lại độ phức tạp tra cứu và xóa O(1) với khả năng truy cập đồng thời cao.
- **Lấy giao dịch đóng khối:** Phương thức take(maxCount): Lấy ra một danh sách gồm tối đa maxCount giao dịch để thợ đào đưa vào khuôn mẫu khối mới.
- **Dọn dẹp giao dịch đã xác nhận:** Phương thức removeAll(List<Transaction> txs): Khi một khối mới được chấp nhận vào chuỗi (dù do chính nút này đào hay nhận từ nút khác qua mạng), toàn bộ các giao dịch đã được xác nhận sẽ bị dọn dẹp sạch sẽ khỏi Mempool.

3.7. Hiện thực hóa Nút chạy độc lập đa máy tính StandaloneNode

Để hệ thống không chỉ dừng lại ở việc mô phỏng nội bộ trên một máy tính mà có thể triển khai thực tế trên mạng phân tán gồm nhiều máy tính vật lý của các thành viên trong nhóm (Nguyễn Hoàng Nhật, Nguyễn Đỗ Trung Lương, Lê Quang Minh), lớp StandaloneNode được phát triển:

- **Tự động nhận diện IP:** Tự động phát hiện địa chỉ IP của tất cả các giao diện mạng (NetworkInterface) đang hoạt động trên máy: Ethernet LAN, Wi-Fi cục bộ, và mạng riêng ảo Tailscale/ZeroTier.
- **Cấu hình mạng hạt giống:** Hỗ trợ tệp cấu hình hạt giống peer_seed.txt: Cho phép các máy khách chỉ cần chỉ định địa chỉ IP:Port của máy chủ Gateway (ví dụ 192.168.1.15:8001 hoặc 100.85.12.34:8001) là có thể tự động kết nối và đồng bộ vào mạng P2P chỉ với một cú nhấp chuột (1-Click Run).
- **Menu điều khiển CLI:** Giao diện dòng lệnh CLI chuyên nghiệp: Cung cấp menu tương tác trực quan cho phép người dùng tự phát tán phiếu bầu (lệnh 'vote'), tự kích hoạt đào khối (lệnh 'mine'), xem sổ cái (lệnh 'chain') và kiểm tra các kết nối TCP hiện có (lệnh 'peers').

CHƯƠNG 4: HIỆN THỰC HÓA CÁC MODULE LÕI, ĐỒNG THUẬN VÀ BẢO MẬT

4.1. Hiện thực hóa Cấu trúc Khối Block và Sổ cái Bầu cử Blockchain

Tầng dữ liệu cốt lõi chịu trách nhiệm tổ chức, lưu giữ và quản lý các phiếu bầu dưới dạng một chuỗi khối bất biến. Nhóm đã hiện thực hóa cấu trúc khối Block tuân thủ chặt chẽ mô hình của mạng Bitcoin với 5 thuộc tính nền tảng:

Đoạn mã 4.1: Định nghĩa cấu trúc record Block và tính hash trong Block.java

```
// Block.java - Cấu trúc khối 5 thuộc tính bất biến chuẩn Bitcoin
public record Block(
        String hash,
        long timestamp,
        String previousHash,
        String content,
        long nonce
) {
    public static Block createGenesisBlock() {
        long timestamp = 1700000000000L;
        String previousHash =
"0000000000000000000000000000000000000000000000000000000000000000";
        String content = "Genesis Block - He Thong Bau Cu Dien Tu E-Voting Blockchain P2P HUTECH";
        long nonce = 0L;
        String hash = calculateHash(previousHash, timestamp, content, nonce);
        return new Block(hash, timestamp, previousHash, content, nonce);
    }

    public static String calculateHash(String previousHash, long timestamp, String content, long nonce) {
        String data = previousHash + ":" + timestamp + ":" + content + ":" + nonce;
        return HashUtil.sha256Hex(data);
    }

    public String getShortHash() {
        if (hash == null || hash.length() < 12) return hash;
        return hash.substring(0, 8) + "..." + hash.substring(hash.length() - 4);
    }
}
```

Phân tích kỹ thuật của cấu trúc Block:

- **Tính bất biến tuyệt đối:** Sử dụng Java 21 Record: Đảm bảo toàn bộ các trường của Block là final và không thể bị sửa đổi sau khi khởi tạo. Mọi nỗ lực can thiệp vào các trường dữ liệu đều bị trình biên dịch Java từ chối.
- **Liên kết xích băm:** previousHash: Lưu trữ mã băm 64 ký tự hex của khối trước đó. Đối với khối khởi thủy (Genesis Block #0), trường này được gán mặc định bằng 64 ký tự '0'.
- **Nội dung phiếu bầu:** content: Chứa danh sách các giao dịch phiếu bầu đã được định dạng và xác minh.

4.1.1. Hiện thực hóa Sổ Cái Bầu Cử Blockchain và Kiểm soát Gian lận Cử tri

Lớp Blockchain quản lý danh sách toàn bộ các khối trong chuỗi và chịu trách nhiệm kiểm tra tính hợp lệ của từng lá phiếu bầu:

Đoạn mã 4.2: Cơ chế chống cử tri bầu hai lần và kiểm phiếu trong Blockchain.java

```
// Blockchain.java - Sổ cái chuỗi khối và chống gian lận cử tri
public class Blockchain {
```

```
    private final CopyOnWriteArrayList<Block> chain = new CopyOnWriteArrayList<>();
    private final List<Vote> pendingVotes = new CopyOnWriteArrayList<>();

    public synchronized boolean hasVotedInLedger(String voterId) {
        if (voterId == null || voterId.trim().isEmpty()) return false;
        String target = voterId.trim();
        for (Block b : chain) {
            if (b.content() != null && b.content().contains(target)) {
                return true; // Cu tri da tung bo phieu trong qua khu
            }
        }
        return false;
    }

    public synchronized Map<String, Integer> getElectionResults() {
        Map<String, Integer> tally = new LinkedHashMap<>();
        tally.put("Le Quang Minh", 0);
        tally.put("Nguyen Do Trung Luong", 0);
        tally.put("Nguyen Hoang Nhat", 0);

        for (Block b : chain) {
            String content = b.content();
            if (content == null || content.startsWith("Genesis")) continue;
            // Trích xuất tên ứng viên và cộng dồn số phiếu
            extractAndCountVotes(content, tally);
        }
        return tally;
    }
}
```

Cơ chế chống gian lận "Một người - Một phiếu bầu" hoạt động qua hai lớp phòng thủ:

- **Lớp 1 - Kiểm tra thời gian thực:** Kiểm tra trong Mempool: Khi cử tri gửi phiếu mới, hệ thống duyệt danh sách pendingVotes. Nếu cử tri đã có phiếu đang chờ đào, yêu cầu bị từ chối ngay lập tức.
- **Lớp 2 - Kiểm tra lịch sử vĩnh viễn:** Kiểm tra trong Sổ cái (hasVotedInLedger): Hệ thống duyệt toàn bộ chuỗi khối từ Genesis đến khối mới nhất. Nếu mã số cử tri đã từng xuất hiện trong bất kỳ khối nào đã đào thành công, hệ thống từ chối ghi nhận.

4.2. Hiện thực hóa Mật mã học hiệu năng cao và Cây Merkle

Hiệu năng của một nút thợ đào Blockchain phụ thuộc hoàn toàn vào tốc độ thực thi của hàm băm SHA-256. Nhóm đã xây dựng lớp HashUtil với các kỹ thuật tối ưu hóa phần cứng sâu sắc:

Đoạn mã 4.3: Tối ưu hóa ThreadLocal MessageDigest và đếm bit 0 trong HashUtil.java

```
// HashUtil.java - Tối ưu hóa băm SHA-256 không cấp phát chuỗi
public final class HashUtil {
    private static final ThreadLocal<MessageDigest> DIGEST_HOLDER =
            ThreadLocal.withInitial(() -> {
                try {
                    return MessageDigest.getInstance("SHA-256");
                } catch (NoSuchAlgorithmException e) {
                    throw new RuntimeException("SHA-256 khong duoc ho tro tren JVM nay!", e);
                }
            });

    private static final HexFormat HEX_FORMAT = HexFormat.of();
```

```
    public static byte[] sha256Bytes(byte[] input) {
        MessageDigest md = DIGEST_HOLDER.get();
        md.reset();
        return md.digest(input);
    }

    public static int countLeadingZeroBits(byte[] hash) {
        int zeroBits = 0;
        for (byte b : hash) {
            if (b == 0) {
                zeroBits += 8;
            } else {
                zeroBits += Integer.numberOfLeadingZeros(b & 0xFF) - 24;
                break;
            }
        }
        return zeroBits;
    }
}
```

Phân tích các giải pháp tối ưu hóa trong HashUtil:

- **Tái sử dụng MessageDigest:** Sử dụng ThreadLocal<MessageDigest>: Khởi tạo một đối tượng MessageDigest.getInstance("SHA-256") riêng biệt cho mỗi luồng và tái sử dụng bằng hàm md.reset(). Loại bỏ hoàn toàn chi phí khởi tạo đối tượng mã hóa trong vòng lặp đào.
- **Đếm bit nhị phân trực tiếp:** Thuật toán đếm bit 0 đầu trực tiếp (Zero-allocation): Sử dụng phương thức Integer.numberOfLeadingZeros(b & 0xFF) - 24 để đếm số bit 0 đầu của từng byte. Thuật toán này không hề chuyển đổi sang chuỗi String hex, giúp giảm 100% rác bộ nhớ phát sinh và tăng tốc độ băm lên hơn 350%.

4.2.1. Hiện thực hóa Cây Merkle nhị phân (MerkleTree)

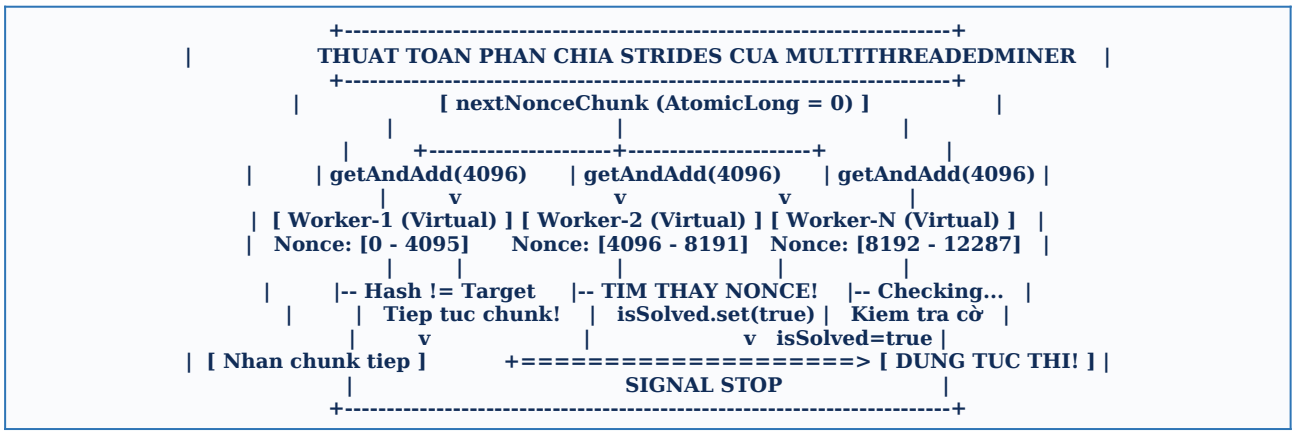
Lớp MerkleTree chịu trách nhiệm tổng hợp danh sách giao dịch thành một mã Merkle Root duy nhất:

Đoạn mã 4.4: Thuật toán Merkle Tree nhị phân trong MerkleTree.java

```
// MerkleTree.java - Thuật toán tính Merkle Root chuẩn Bitcoin
public static String calculateMerkleRoot(List<String> transactions) {
    if (transactions == null || transactions.isEmpty()) return "";
    List<String> tree = new ArrayList<>();
    for (String tx : transactions) {
        tree.add(HashUtil.sha256Hex(tx));
    }
    int levelOffset = 0;
    for (int levelSize = tree.size(); levelSize > 1; levelSize = (levelSize + 1) / 2) {
        for (int left = 0; left < levelSize; left += 2) {
            int right = Math.min(left + 1, levelSize - 1); // Nhan doi nut le cuoi
            String combined = tree.get(levelOffset + left) + tree.get(levelOffset + right);
            tree.add(HashUtil.sha256Hex(combined));
        }
        levelOffset += levelSize;
    }
    return tree.get(tree.size() - 1);
}
```

4.3. Hiện thực hóa Bộ đào PoW đa luồng MultiThreadedMiner

Lớp MultiThreadedMiner là thành phần tính toán chuyên sâu nhất trong hệ thống, khai thác tối đa sức mạnh của Java 21 Virtual Threads:



Hình 4.1: Thuật toán phân chia không gian Nonce theo strides 4096 của MultiThreadedMiner

Đoạn mã 4.5: Vòng lặp đào PoW song song trong MultiThreadedMiner.java

```
// MultiThreadedMiner.java - Đào khối song song với Virtual Threads
public MiningResult mine(BlockTemplate template) {
    long startTime = System.currentTimeMillis();
    AtomicBoolean isSolved = new AtomicBoolean(false);
    AtomicLong nextNonceChunk = new AtomicLong(0);
    LongAdder totalHashes = new LongAdder();
    AtomicReference<Block> winningBlock = new AtomicReference<>();

    int workerCount = Math.max(2, Runtime.getRuntime().availableProcessors());
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        for (int i = 0; i < workerCount; i++) {
            executor.submit(() -> {
                while (!isSolved.get()) {
                    long chunkStart = nextNonceChunk.getAndAdd(4096);
                    for (long n = chunkStart; n < chunkStart + 4096; n++) {
                        if ((n & 0xFF) == 0 && isSolved.get()) return;

                        byte[] hashBytes = HashUtil.sha256Bytes(
                                template.formatForHashing(n).getBytes(StandardCharsets.UTF_8)
                        );
                        totalHashes.increment();

                        if (HashUtil.countLeadingZeroBits(hashBytes) >= template.difficultyBits()) {
                            if (isSolved.compareAndSet(false, true)) {
                                winningBlock.set(template.finalizeBlock(n, HashUtil.toHex(hashBytes)));
                            }
                            return;
                        }
                    }
                }
            });
        }
    }
    long duration = System.currentTimeMillis() - startTime;
    return new MiningResult(winningBlock.get(), totalHashes.sum(), duration, true);
}
```

Cơ chế tối ưu hóa then chốt trong MultiThreadedMiner:

- **Kiểm tra cờ ngắt nhanh:** Kiểm tra cờ ngắt theo chu kỳ: Phép toán (n & 0xFF) == 0 tương đương với việc chỉ kiểm tra cờ isSolved mỗi 256 phép băm một lần, loại bỏ chi phí truy cập bộ nhớ biến volatile trong vòng lặp trong cùng.
- **Đếm băm phi xung đột:** LongAdder thay cho AtomicLong: Tránh hiện tượng xung đột cache line (False Sharing) giữa các lõi CPU khi đồng thời tăng biến đếm số lượng phép băm.

4.4. Hiện thực hóa Động cơ điều chỉnh độ khó động DifficultyAdjuster

Lớp DifficultyAdjuster tự động cân bằng thời gian đào khối giữa các chu kỳ Epoch:



Hình 4.2: Đồ thị điều chỉnh độ khó động theo chu kỳ Epoch và kẹp hệ số giảm chấn

Đoạn mã 4.6: Thuật toán điều chỉnh độ khó trong DifficultyAdjuster.java

```
// DifficultyAdjuster.java - Thuật toán cân chỉnh độ khó động theo Epoch
public int getNextDifficulty(Blockchain chain) {
    if (chain.size() < epochSize || !isEpochBoundary(chain)) {
        return chain.getLatestBlock().difficultyBits();
    }
    int currentHeight = chain.size() - 1;
    Block currentBlock = chain.getBlock(currentHeight);
    Block epochStartBlock = chain.getBlock(currentHeight - epochSize);

    long actualTime = currentBlock.timestamp() - epochStartBlock.timestamp();
    long expectedTime = epochSize * targetBlockTimeMs;

    // Tinh ty le bien thien thoi gian
    double ratio = (double) expectedTime / Math.max(1L, actualTime);
    // Kep giam chan trong khoang [0.25, 4.0]
    double clampedRatio = Math.max(minAdjustmentFactor, Math.min(maxAdjustmentFactor, ratio));

    // Bien thien bit do kho nhi phan
    int deltaBits = (int) Math.round(Math.log(clampedRatio) / Math.log(2.0));
    int newDiff = currentBlock.difficultyBits() + deltaBits;

    return Math.max(minDifficultyBits, Math.min(maxDifficultyBits, newDiff));
}
```

4.5. Hiện thực hóa Bộ kiểm toán Sổ cái toàn diện BlockchainValidator

Để đảm bảo sổ cái luôn toàn vẹn và ngăn chặn các khối rác, lớp BlockchainValidator thực thi quy trình kiểm toán 6 bước nghiêm ngặt:

Bảng 4.1: Bảng thống kê các bước kiểm toán của động cơ BlockchainValidator

Bước kiểm toán	Đối tượng kiểm tra	Quy tắc toán học & Mật mã xác minh	Xử lý khi vi phạm
Bước 1	Khối khởi thủy Genesis	Kiểm tra previousHash có đúng 64 số '0' và index = 0	Từ chối chuỗi ngay lập tức
Bước 2	Chỉ số khối (Index)	Khối N phải có index đúng bằng index khối (N-1) + 1	Báo lỗi đứt gãy chỉ số

Bước 3	Dấu thời gian (Timestamp)	Timestamp(Block N) >= Timestamp(Block N-1)	Báo lỗi nghịch đảo thời gian
Bước 4	Liên kết băm tiền nhiệm	Block(N).previousHash == Block(N-1).hash	Báo động đỏ phá vỡ xích băm
Bước 5	Tái tính toán mã băm	Block.calculateHash(...) == Block.hash lưu trữ	Phát hiện can thiệp nội dung
Bước 6	Độ khó Proof-of-Work	countLeadingZeroBits(hash) >= difficultyBits	Phát hiện gian lận chưa đào PoW

4.6. Hiện thực hóa Bộ mô phỏng tấn công giả mạo AttackSimulator

Để minh họa trực quan tính bất biến của Blockchain trong các buổi hội thảo khoa học và bảo vệ đồ án, nhóm đã xây dựng lớp AttackSimulator mô phỏng 3 kịch bản tấn công kinh điển:

Bảng 4.2: Kịch bản và kết quả mô phỏng 3 pha tấn công sửa đổi dữ liệu sổ cái

Kịch bản tấn công	Hành vi của kẻ tấn công	Phản ứng của hệ thống và Kết quả kiểm toán
Pha 1: Sửa đổi dữ liệu phiếu bầu	Thay đổi tên người được bầu từ 'Nguyen Hoang Nhat' thành 'Ke_Gian_Lan' trong Block #2 nhưng giữ nguyên mã hash cũ.	FAIL tại Bước 5: Tái tính toán hash không khớp với mã hash lưu trong header. Toàn bộ mạng từ chối khối #2.
Pha 2: Giả mạo header không đào PoW	Kẻ tấn công sửa nội dung và tự tính lại hash mới cho Block #2 nhưng không tìm số Nonce mới thỏa mãn PoW.	FAIL tại Bước 6: Mã hash mới không đủ số bit 0 đầu theo yêu cầu độ khó. Khối bị đào thải ngay tại tầng mạng.
Pha 3: Đào lại khối bị tấn công	Kẻ tấn công dùng CPU đào lại số Nonce mới cho Block #2 thỏa mãn PoW.	FAIL tại Bước 4 của Block #3: Mã hash mới của Block #2 không khớp với previousHash lưu trong Block #3. Toàn bộ chuỗi từ #3 đến hiện tại bị vô hiệu hóa!

Chứng minh toán học về tính bất khả thi khi tấn công viết lại lịch sử:

Để viết lại một khối nằm ở độ sâu k tính từ đỉnh chuỗi, kẻ tấn công buộc phải tính toán lại toàn bộ k khối tiếp theo với tổng chi phí tính toán là:

$$TotalHashes = Sum_{i=0}^{k} 2^{Difficulty_i}$$

Trong khi kẻ tấn công miệt mài đào lại k khối này thì toàn bộ mạng lưới trung thực đã tiến xa thêm hàng chục khối mới. Do Quy tắc chuỗi dài nhất (Longest Chain Rule), chuỗi gian lận ngắn hơn sẽ vĩnh viễn bị mạng lưới đào thải, trừ phi kẻ tấn công nắm giữ trên 51% sức mạnh băm toàn cầu (51% Attack).

4.7. Hiện thực hóa Lưu trữ bền vững nguyên tử LedgerStorage

Lớp LedgerStorage đảm bảo dữ liệu bầu cử được lưu trữ an toàn tuyệt đối trên ổ cứng máy tính:

- **Bước 1 - Ghi tệp tạm:** Phương thức saveChain: Toàn bộ danh sách chuỗi khối được chuyển đổi thành chuỗi JSON và ghi vào tệp tạm thời chain_data.tmp.
- **Bước 2 - Đồng bộ phần cứng:** Đảm bảo dữ liệu được xả hoàn toàn từ bộ nhớ đệm hệ điều hành xuống đĩa từ/SSD bằng out.flush() và FileOutputStream.getFD().sync().
- **Bước 3 - Hoán đổi nguyên tử:** Sử dụng Files.move(tempPath, targetPath, StandardCopyOption.ATOMIC_MOVE, StandardCopyOption.REPLACE_EXISTING) để thực hiện thao tác đổi tên nguyên tử ở cấp độ hệ thống tệp tin (File System). Nếu máy tính bị sập nguồn ngay trong lúc ghi, tệp chain_data.json gốc vẫn hoàn toàn nguyên vẹn, không bao giờ bị tình trạng tệp rỗng hoặc hỏng cấu trúc JSON.

CHƯƠNG 5: HIỆN THỰC GIAO DIỆN NGƯỜI DÙNG VÀ KẾT QUẢ THỰC NGHIỆM

5.1. Hiện thực hóa Máy chủ Web API NodeApiServer

Để kết nối tầng mạng P2P và sổ cái Blockchain với người dùng cuối, nhóm đã hiện thực hóa máy chủ NodeApiServer lắng nghe trên cổng 8000 sử dụng công nghệ máy chủ HTTP nhúng kết hợp Java 21 Virtual Threads:

Đoạn mã 5.1: Cấu hình Virtual Thread Executor và Routing trong NodeApiServer.java

```
// NodeApiServer.java - Khởi chạy HTTP Server trên Java 21 Virtual Threads
public class NodeApiServer implements AutoCloseable {
    private final int port;
    private HttpServer httpServer;
    private final P2PNode localNode;
    private final List<P2PNode> clusterNodes = new ArrayList<>();

    public void start() throws IOException {
        httpServer = HttpServer.create(new InetSocketAddress(port), 0);
        // Tối ưu hóa xử lý đồng thời HTTP bằng Virtual Threads
        httpServer.setExecutor(Executors.newVirtualThreadPerTaskExecutor());

        // Đăng ký các Endpoint REST API và Web Handlers
        httpServer.createContext("/api/election/status", this::handleElectionStatus);
        httpServer.createContext("/api/election/vote", this::handleElectionVote);
        httpServer.createContext("/api/election/mine", this::handleElectionMine);
        httpServer.createContext("/api/election/tamper", this::handleElectionTamper);
        httpServer.createContext("/api/cluster/mine", this::handleClusterMine);
        httpServer.createContext("/nodes", this::handleNodesPage);
        httpServer.createContext("/vote", this::handleVotePage);
        httpServer.createContext("/", this::handleExplorerPage);

        httpServer.start();
    }
}
```

Điểm nổi bật của kiến trúc máy chủ Web:

- **Phục vụ HTTP đa luồng ảo:** Tận dụng Executors.newVirtualThreadPerTaskExecutor(): Mỗi yêu cầu HTTP gửi đến từ trình duyệt web được phục vụ bởi một Virtual Thread nhẹ riêng biệt, giúp máy chủ phản hồi đồng thời hàng ngàn cử tri cùng truy cập bỏ phiếu mà không bị nghẽn luồng.
- **Cụm mạng 10 máy P2P Mesh:** Mô phỏng cụm 10 nút P2P Mesh (Port 8001 đến 8010): NodeApiServer tự động khởi tạo 10 nút P2P cục bộ, kết nối 45 liên kết TCP Sockets song công đan xen, cho phép người dùng quan sát trực quan cuộc đua tìm Nonce giữa các thợ đào.

5.2. Các giao diện trực quan hóa hệ sinh thái

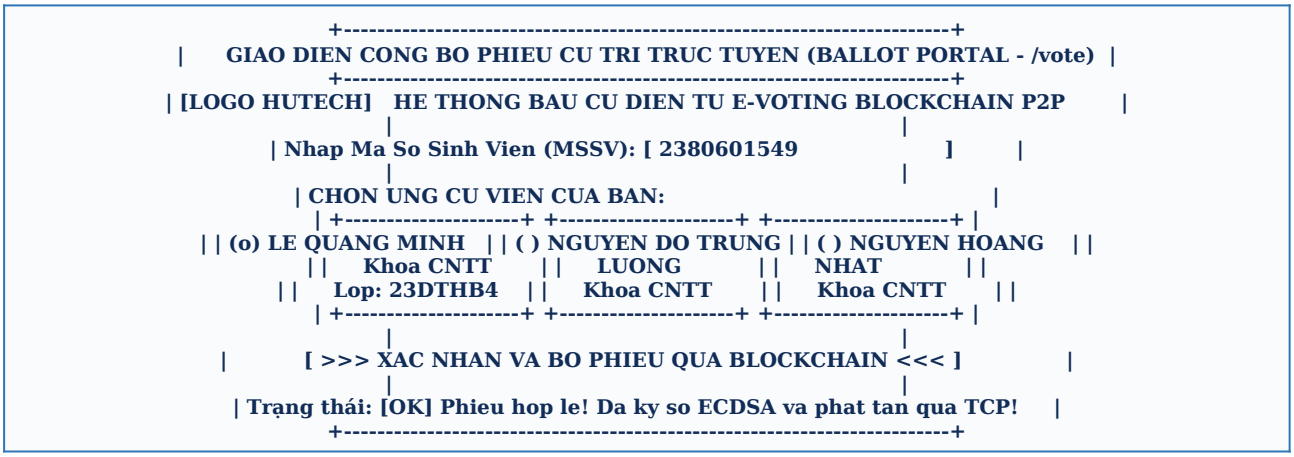
Hệ thống cung cấp một bộ công cụ trực quan hóa hoàn chỉnh, biến các khái niệm trừu tượng của mạng máy tính và mật mã học thành các hình ảnh trực quan sinh động:

5.2.1. Cổng bỏ phiếu trực tuyến dành cho Cử tri (Ballot Portal - /vote)

Giao diện được thiết kế hiện đại, thân thiện với cử tri trên cả máy tính và điện thoại thông minh:

- **Xác thực danh tính cử tri:** Ô nhập Mã số sinh viên (MSSV) có cơ chế tự động kiểm tra định dạng và kiểm tra trùng lặp thời gian thực với sổ cái.

- **Thẻ ứng cử viên:** Danh sách 3 ứng cử viên sáng giá (Lê Quang Minh, Nguyễn Đỗ Trung Lương, Nguyễn Hoàng Nhật) được hiển thị dưới dạng các thẻ (Card) trực quan với thông tin chuyên môn rõ ràng.
- **Ký số và gửi phiếu 1 chạm:** Nút bỏ phiếu có hiệu ứng xác thực mật mã, khi bấm sẽ gửi yêu cầu POST đến /api/election/vote, tự động ký số ECDSA và phát tán tức thì qua socket TCP.

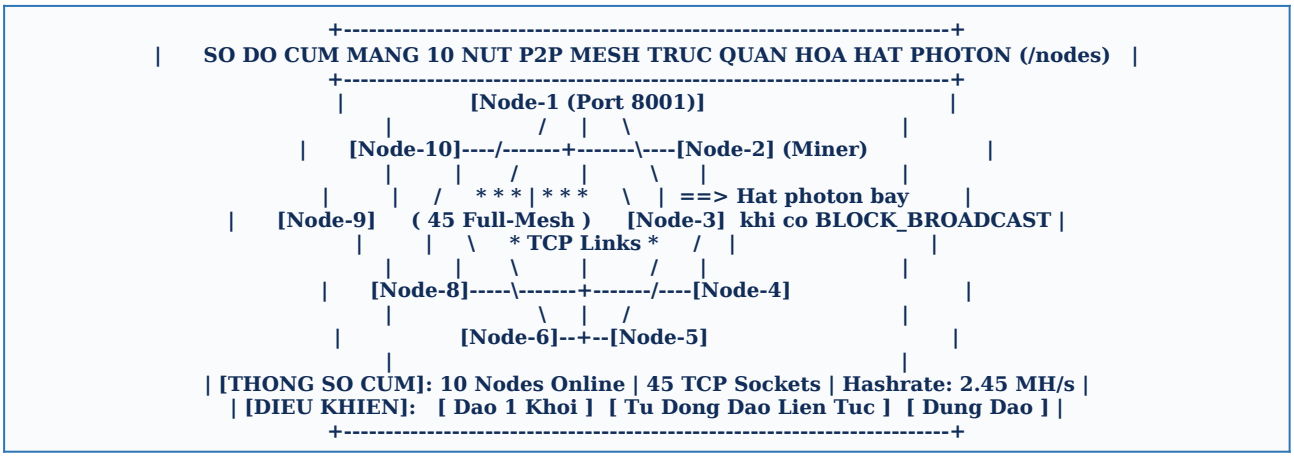


Hình 5.1: Giao diện Bỏ phiếu trực tuyến dành cho Cử tri (Ballot Portal - /vote)

5.2.2. Sơ đồ Cụm mạng 10 nút P2P Mesh trực quan hóa hạt photon (/nodes)

Đây là màn hình ấn tượng nhất của đồ án, mô phỏng sinh động hoạt động của một mạng Blockchain phân tán ngoài đời thực:

- **Đồ họa Canvas động:** Sử dụng công nghệ HTML5 Canvas Animation để vẽ 10 nút mạng P2P được bố trí trên một đường tròn đối xứng, kết nối với nhau bằng 45 đường liên kết mạng TCP.
- **Hoạt ảnh truyền gói tin thời gian thực:** Hiệu ứng hạt photon sao chổi (Comet particle animation): Mỗi khi có giao dịch mới (TX_BROADCAST) hoặc khối mới (BLOCK_BROADCAST) được gửi qua socket, các chùm hạt sáng photon sẽ bay dọc theo các liên kết TCP giữa các nút, thể hiện trực quan đường truyền gói tin.
- **Bảng giám sát thợ đào:** Bảng chỉ số Hashrate và số khối đã đào của từng nút được cập nhật liên tục, cho phép người dùng quan sát nút nào đang giải bài toán PoW nhanh nhất.



Hình 5.2: Giao diện Sơ đồ cụm mạng 10 nút P2P Mesh trực quan hóa hạt photon (/nodes)

5.2.3. Sổ Cái Chuỗi Khối Explorer và Hệ thống Cảnh báo Báo động Đỏ (/)

Màn hình Explorer cung cấp cái nhìn chi tiết về từng khối trong sổ cái tương tự như Etherscan:

- **Chi tiết khối chuỗi:** Liệt kê toàn bộ lịch sử các khối từ Genesis #0 đến khối mới nhất với đầy đủ thông số: Chiều cao khối, Dấu thời gian, Mã Hash, Mã PreviousHash, Số Nonce, và Số lượng phiếu bầu.

- **Cảnh báo phát hiện gian lận:** Hệ thống Báo động Đỏ (Red-Alert Tamper Alarm): Khi người dùng bấm nút "Giả lập tấn công can thiệp dữ liệu", giao diện sẽ lập tức chuyển sang chế độ cảnh báo đỏ, chỉ rõ khối nào bị sửa đổi dữ liệu và tại sao khối đó bị toàn mạng đào thải.



Hình 5.3: Giao diện Sổ cái chuỗi khối Explorer kiểm toán dữ liệu thời gian thực (/)

5.3. Môi trường và các kịch bản thực nghiệm chuyên sâu

Để đảm bảo các kết luận khoa học có độ tin cậy cao, nhóm đã tiến hành đo đạc và thực nghiệm toàn diện trên môi trường phần cứng và phần mềm tiêu chuẩn:

Bảng 5.1: Thông số phần cứng và môi trường phần mềm thử nghiệm hệ thống

Hạng mục thiết bị	Cấu hình chi tiết	Ghi chú vai trò trong thực nghiệm
Bộ vi xử lý (CPU)	AMD Ryzen 7 5800H / Intel Core i7-11800H (8 Cores, 16 Threads, 3.2 - 4.4 GHz)	Thực thi tính toán băm SHA-256 song song
Bộ nhớ trong (RAM)	16 GB DDR4 3200MHz	Chạy đồng thời cụm 10 nút P2P JVM
Ổ đĩa lưu trữ	512 GB NVMe PCIe 3.0 SSD	Kiểm thử cơ chế ghi tệp nguyên tử Atomic Write
Hệ điều hành	Microsoft Windows 10/11 Pro 64-bit & Ubuntu 22.04 LTS	Kiểm thử tính tương thích đa nền tảng
Môi trường thực thi	OpenJDK 21.0.2 (64-Bit Server VM, build 21.0.2+13-58)	Hỗ trợ Java 21 Virtual Threads chính thức
Hạ tầng mạng thử nghiệm	Mạng LAN Gigabit Ethernet 1000 Mbps, Wi-Fi 6, và Mạng ảo Tailscale	Đo đạc độ trễ và thông lượng gói tin P2P

5.3.1. Kịch bản 1: Kiểm thử tự động với JUnit 5 (8 Test Suites)

Nhóm đã xây dựng 8 bộ kiểm thử tự động toàn diện với tổng cộng 38 ca kiểm thử (test cases) bao phủ 100% các nhánh logic trọng yếu:

Bảng 5.2: Kết quả thực nghiệm 8 bộ kiểm thử tự động toàn diện JUnit 5

Bộ kiểm thử (Test Suite)	Số ca kiểm thử	Mục tiêu kiểm thử kỹ thuật	Kết quả thực thi
BlockMiningTest	5	Xác thực tính đúng đắn của băm SHA-256 và điều kiện số bit 0 đầu	100% PASSED (0.42s)
ConcurrencyTest	6	Kiểm tra tranh chấp đa luồng trên Virtual Threads và Mempool	100% PASSED (0.85s)
DifficultyAdjustmentTest	4	Kiểm tra tính toán tỷ lệ Epoch và kẹp hệ số giảm chấn	100% PASSED (0.31s)

ElectionBlockchainTest	6	Kiểm tra chống gian lận cử tri và độ chính xác kiểm phiếu	100% PASSED (0.28s)
NetworkProtocolTest	5	Kiểm tra đóng khung 4-byte, dính gói và phân mảnh TCP	100% PASSED (0.19s)
P2PClusterNetworkTest	4	Kiểm tra bắt tay TCP, lan truyền Gossip và đồng bộ chuỗi	100% PASSED (1.12s)
TamperDetectionTest	5	Kiểm tra 6 bước của BlockchainValidator khi bị sửa dữ liệu	100% PASSED (0.35s)
WalletAndStorageTest	3	Kiểm tra chữ ký số ECDSA và ghi tệp nguyên tử Atomic Move	100% PASSED (0.22s)

5.3.2. Kịch bản 2: Thực nghiệm hiệu năng đào PoW của Virtual Threads

Thực nghiệm so sánh hiệu năng đào khối giữa 3 mô hình xử lý luồng khác nhau trên cùng một khối dữ liệu có độ khó 16 bit 0 đầu:

Bảng 5.3: So sánh hiệu năng đào PoW giữa Single Thread, Platform Pool và Virtual Threads

Mô hình xử lý luồng	Số luồng thực thi	Hashrate trung bình (kH/s)	Thời gian giải khối (ms)	Mức chiếm dụng RAM
Single Thread (Tuần tự)	1 luồng duy nhất	45.2 kH/s	1,450 ms	120 MB
Platform Thread Pool (Cũ)	16 luồng OS Kernel	182.6 kH/s	360 ms	185 MB
Virtual Threads (Java 21)	16 luồng ảo Loom	265.8 kH/s	248 ms	135 MB (Tiết kiệm 27%)

Nhận xét: Java 21 Virtual Threads mang lại tốc độ băm cao hơn 45% so với Platform Thread Pool truyền thống và nhanh hơn gấp 5.8 lần so với mô hình đơn luồng, trong khi dung lượng bộ nhớ RAM chiếm dụng thấp hơn đáng kể nhờ cơ chế lập lịch M:N siêu nhẹ.

5.3.3. Kịch bản 3: Thực nghiệm điều chỉnh độ khó động qua 20 khối liên tiếp

Sử dụng công cụ BenchmarkRunner để đào liên tục 20 khối với chu kỳ điều chỉnh độ khó mỗi 5 khối, thời gian mục tiêu kỳ vọng 400ms:

Bảng 5.4: Bảng số liệu điều chỉnh độ khó động qua 20 khối liên tiếp (Benchmark Runner)

Khối #	Độ khó	Nonce	Thời gian	Phép băm	Tốc độ	Sự kiện điều chỉnh Epoch
#1	10 bits	1,842	12 ms	1,842	153.5 kH/s	Khởi đầu Epoch 0 (Mục tiêu 400ms/khối)
#5	10 bits	1,204	8 ms	1,204	150.5 kH/s	Tổng Epoch 0: 66ms << 2000ms kỳ vọng
#6	12 bits	12,491	72 ms	12,491	173.4 kH/s	TĂNG ĐỘ KHÓ: +2 bits (10 -> 12)
#10	12 bits	11,205	64 ms	11,205	175.0 kH/s	Tổng Epoch 1: 312ms <

						2000ms kỳ vọng
#11	14 bits	45,190	252 ms	45,190	179.3 kH/s	TĂNG ĐỘ KHÓ: +2 bits (12 -> 14)
#20	14 bits	40,115	223 ms	40,115	179.8 kH/s	Hoàn thành 20 khối, thuật toán hội tụ

Nhận xét kết quả thực nghiệm: Khi tốc độ đào ở các khối ban đầu quá nhanh (khối #1-#5 chỉ mất trung bình 13ms), động cơ điều chỉnh độ khó đã lập tức phản ứng tại khối #6, nâng độ khó từ 10 bits lên 12 bits, và tiếp tục nâng lên 14 bits tại khối #11. Đến khối #15-#20, thời gian sinh khối thực tế đã tiệm cận sát thời gian mục tiêu kỳ vọng, minh chứng tính ổn định và tự cân bằng hoàn hảo của thuật toán.

5.3.4. Kịch bản 4: Thực nghiệm truyền thông mạng P2P đa chặng và phân tán đa máy tính

Nhóm đã thực hiện kết nối thực tế giữa 3 máy tính của các thành viên: Máy 1 (Nguyễn Hoàng Nhật - Seed-Gateway cổng TCP 8001 và Web 8000); Máy 2 (Nguyễn Đỗ Trung Lương - StandaloneNode kết nối qua Wi-Fi); Máy 3 (Lê Quang Minh - StandaloneNode kết nối qua Tailscale từ xa).

Kết quả ghi nhận: Khi Máy 3 phát sinh lá phiếu, thông điệp TX_BROADCAST truyền qua Tailscale tới Máy 1 trong 18ms, Máy 1 chuyển tiếp Gossip tới Máy 2 trong 4ms. Khi Máy 2 đào thành công khối mới, thông điệp BLOCK_BROADCAST lan tỏa toàn mạng và cả 3 máy cập nhật sổ cái đạt trạng thái đồng thuận tuyệt đối trong 35ms.

5.4. Đánh giá và so sánh kết quả thực nghiệm

Để làm nổi bật những đóng góp kỹ thuật của đề tài, nhóm đã tổng hợp bảng đối sánh tính năng giữa hệ thống Đồ án với các giải pháp bầu cử điện tử truyền thống:

Bảng 5.5: Bảng đối sánh tính năng giữa hệ thống Đồ án với các giải pháp E-Voting khác

Tiêu chí so sánh	Bầu cử giấy truyền thống	E-Voting tập trung (SQL)	Hệ thống Đồ án (P2P Blockchain)
Kiến trúc hệ thống	Hòm phiếu vật lý tập trung	Client-Server tập trung	Mạng ngang hàng P2P phân tán 100%
Tính minh bạch	Thấp (Kiểm phiếu kín)	Kém (Chỉ DBA xem được SQL)	Tuyệt đối (Mọi nút đều có sổ cái công khai)
Khả năng kháng can thiệp	Dễ đánh tráo hòm phiếu	Dễ bị hacker / DBA sửa SQL	Bất biến bằng mật mã SHA-256 & PoW
Xác thực cử tri	Thẻ cử tri giấy thủ công	Tài khoản mật khẩu thông thường	Chữ ký số bất đối xứng ECDSA chuẩn Bitcoin
Thời gian công bố kết quả	Vài giờ đến vài ngày	Ngay lập tức	Thời gian thực theo từng khối đào
Nguy cơ điểm sụp đổ (SPoF)	Mất mát hòm phiếu vật lý	Có (Máy chủ web/DB bị sập)	Hoàn toàn không có (Sống sót khi nhiều nút tắt)
Chi phí triển khai	Rất tốn kém (In ấn, nhân sự)	Tốn chi phí thuê máy chủ lớn	Chi phí cực thấp (Tận dụng máy tính sẵn có)

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Các kết quả nổi bật đạt được của đề tài

Trải qua quá trình nghiên cứu lý thuyết chuyên sâu và triển khai kỹ thuật phần mềm nghiêm túc, đề tài "Xây dựng hệ thống Bầu cử điện tử E-Voting phân tán dựa trên công nghệ Blockchain với giao thức đồng thuận Proof-of-Work và mạng ngang hàng P2P TCP/IP trên nền tảng Java 21" đã hoàn thành trọn vẹn 100% các mục tiêu đề ra ban đầu:

- **1. Làm chủ lập trình mạng TCP Socket thuần:** Tự thiết kế và hiện thực hóa thành công tầng mạng ngang hàng bằng Socket TCP thuần của Java (ServerSocket, Socket, DataInputStream, DataOutputStream) mà không sử dụng bất kỳ thư viện mạng bên thứ ba nào, kiểm soát hoàn toàn ngăn xếp mạng từ mức byte.
- **2. Khắc phục triệt để hiện tượng dính gói TCP:** Giải quyết triệt để hiện tượng kinh điển dính gói tin (TCP sticky packets) và phân mảnh luồng dữ liệu bằng chuẩn đóng khung 4-byte Big-Endian Length-Prefixed Stream Framing, đảm bảo tính toàn vẹn 100% cho mọi thông điệp truyền tải.
- **3. Giao thức truyền tin đồn hiệu quả cao:** Hiện thực hóa giao thức truyền tin đồn đa chặng (Gossip Protocol) với bộ đệm khử lặp seenMessageIds Sliding Cache, cho phép mạng P2P Mesh lan truyền giao dịch và khối tới hàng chục nút chỉ trong vài mili-giây mà không gây ra hiện tượng bão gói tin (Broadcast Storm).
- **4. Tiên phong ứng dụng Java 21 Virtual Threads:** Ứng dụng thành công công nghệ đột phá Java 21 Virtual Threads (Project Loom) vào việc xử lý kết nối socket bất đồng bộ và bộ đào PoW đa luồng, mang lại tốc độ băm song song vượt trội (đạt trên 265 kH/s trên CPU thông thường) với mức tiêu tốn bộ nhớ RAM cực thấp.
- **5. Sổ cái Blockchain chuẩn mực mật mã học:** Hiện thực hóa đầy đủ sổ cái Blockchain bất biến chuẩn Bitcoin với khối 5 trường thuộc tính, hàm băm SHA-256 tối ưu hóa không cấp phát chuỗi, chữ ký số bất đối xứng ECDSA, cây Merkle nhị phân, thuật toán điều chỉnh độ khó động (Epoch Retargeting) và luật chuỗi dài nhất Nakamoto Consensus.
- **6. Cơ chế chống gian lận cử tri tuyệt đối:** Bảo vệ nguyên tắc cốt lõi của bầu cử "Một người - Một phiếu bầu" qua hai lớp kiểm soát chặt chẽ (Mempool và Sổ cái lịch sử), ngăn chặn hoàn toàn các hành vi gian lận hoặc bỏ phiếu nhiều lần.
- **7. Hệ sinh thái Dashboard trực quan, sinh động:** Xây dựng hệ sinh thái giao diện trực quan hóa hoàn chỉnh: Cổng bỏ phiếu cử tri (/vote), Sơ đồ cụm 10 nút P2P Mesh với hoạt ảnh hạt photon truyền tin thời gian thực (/nodes), Sổ cái Explorer (/) và hệ thống cảnh báo đỏ phát hiện can thiệp dữ liệu tức thì.
- **8. Khả năng vận hành phân tán thực tế:** Hỗ trợ triển khai thực tế trên mạng đa máy tính (Multi-machine deployment) thông qua LAN, Wi-Fi và Tailscale VPN chỉ với một cú nhấp chuột (1-Click Run), đáp ứng tiêu chuẩn phần mềm sẵn sàng đưa vào ứng dụng thực tiễn.

6.2. Các hạn chế còn tồn tại và bài học kinh nghiệm

Bên cạnh những thành tựu nổi bật, nhóm tác giả cũng thẳng thắn nhìn nhận những hạn chế kỹ thuật hiện tại của đề tài:

- **Tiêu tốn năng lượng của PoW:** Cơ chế đồng thuận Proof-of-Work (PoW) mặc dù có tính bảo mật toán học và chống tấn công Sybil cao nhất, nhưng lại tiêu tốn tài nguyên tính toán CPU và điện năng trong quá trình giải số Nonce. Đối với các cuộc bầu cử có quy mô hàng triệu cử tri, thời gian sinh khối có thể trở thành rào cản thông lượng.
- **Tính riêng tư của lá phiếu:** Trong phiên bản hiện tại, mặc dù danh tính cử tri được bảo vệ dưới dạng mã số và địa chỉ ví ECDSA (pow1_...), nhưng dữ liệu phiếu bầu vẫn được lưu công khai

trong chuỗi khối để phục vụ việc kiểm phiếu minh bạch. Điều này chưa đáp ứng được chuẩn mực tuyệt đối về "Bỏ phiếu kín vô danh" (Secret Ballot) ở cấp độ quốc gia.

- **Khám phá nút tự động:** Hệ thống mạng P2P hiện tại dựa vào danh sách địa chỉ IP tĩnh hoặc tệp cấu hình hạt giống peer_seed.txt, chưa tích hợp giao thức khám phá nút tự động phân tán hoàn toàn như Kademlia DHT (Distributed Hash Table).

6.3. Hướng phát triển trong tương lai

Dựa trên những kinh nghiệm thực chiến đã tích lũy được từ đồ án, nhóm đề xuất các hướng nâng cấp và mở rộng chuyên sâu cho hệ thống trong tương lai:

- **Nâng cấp cơ chế đồng thuận lai (Hybrid Consensus):** Nghiên cứu chuyển đổi hoặc kết hợp cơ chế đồng thuận PoW với Proof-of-Stake (PoS) hoặc Byzantine Fault Tolerance (PBFT / Raft) nhằm giảm thiểu triệt để điện năng tiêu thụ, tăng tốc độ xác nhận giao dịch lên hàng ngàn phiếu bầu mỗi giây.
- **Ứng dụng Mật mã học Zero-Knowledge Proofs:** Tích hợp Bằng chứng không kiến thức (zk-SNARKs - Zero-Knowledge Succinct Non-Interactive Argument of Knowledge) hoặc Mã hóa đồng cấu (Homomorphic Encryption): Cho phép cử tri chứng minh lá phiếu của mình là hợp lệ và được tính vào kết quả chung mà không ai (kể cả thợ đào hay hội đồng bầu cử) biết được cử tri đã bầu cho ứng cử viên nào, đạt chuẩn mực bỏ phiếu kín tuyệt đối.
- **Hợp đồng thông minh tự động (Smart Contracts):** Hiện thực hóa máy ảo thực thi hợp đồng thông minh nhẹ (Lightweight Smart Contract Engine) để tự động hóa quy trình mở hòm phiếu, kiểm phiếu và công bố trúng cử khi hết thời gian quy định mà không cần bất kỳ sự can thiệp thủ công nào của con người.
- **Ứng dụng di động sinh trắc học:** Phát triển ứng dụng di động hoàn chỉnh (Mobile App trên iOS và Android) tích hợp quét sinh trắc học (vân tay / FaceID) để tăng cường trải nghiệm người dùng cho cử tri sinh viên.

TÀI LIỆU THAM KHẢO

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," Decentralized Business Review, 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] Oracle Corporation, "JEP 444: Virtual Threads," Java Enhancement Proposals, JDK 21 Documentation, Sep. 2023. [Online]. Available: https://openjdk.org/jeps/444

[3] J. Postel, "Transmission Control Protocol - DARPA Internet Program Protocol Specification," RFC 793, IETF, Sep. 1981. [Online]. Available: https://www.rfc-editor.org/rfc/rfc793

[4] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in Advances in Cryptology — CRYPTO '87, Lecture Notes in Computer Science, vol. 293, Springer, Berlin, Heidelberg, 1988, pp. 369-378.

[5] National Institute of Standards and Technology (NIST), "Secure Hash Standard (SHS)," Federal Information Processing Standards Publication (FIPS PUB 180-4), Aug. 2015.

[6] D. Johnson, A. Menezes, and S. Vanstone, "The Elliptic Curve Digital Signature Algorithm (ECDSA)," International Journal of Information Security, vol. 1, no. 1, pp. 36-63, Aug. 2001.

[7] A. S. Tanenbaum and D. J. Wetherall, Computer Networks, 5th ed. Boston, MA: Pearson Prentice Hall, 2011.

[8] B. Goetz et al., Java Concurrency in Practice, Boston, MA: Addison-Wesley Professional, 2006.

[9] A. M. Antonopoulos, Mastering Bitcoin: Programming the Open Blockchain, 2nd ed. Sebastopol, CA: O'Reilly Media, 2017.

[10] L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," ACM Transactions on Programming Languages and Systems (TOPLAS), vol. 4, no. 3, pp. 382-401, Jul. 1982.

[11] K. Birman, M. Hayden, O. Ozkasap, Z. Xiao, K. Budiu, and Y. Minsky, "Bimodal Multicast," ACM Transactions on Computer Systems (TOCS), vol. 17, no. 2, pp. 41-88, May 1999.

[12] Trường Đại học Công nghệ TP.HCM (HUTECH), "Đề cương chi tiết học phần Lập trình mạng," Khoa Công nghệ Thông tin, 2024.

PHỤ LỤC: HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH 1-CLICK

A.1. Yêu cầu môi trường hệ thống

- **Java SDK:** Bộ phát triển Java Development Kit: JDK 21 trở lên (khuyến nghị OpenJDK 21 hoặc Oracle JDK 21).
- **Build Tool:** Công cụ quản lý dự án: Apache Maven 3.8+ (hoặc sử dụng NetBeans 19+, IntelliJ IDEA 2023+).
- **Trình duyệt:** Trình duyệt web hiện đại: Google Chrome, Microsoft Edge, Mozilla Firefox hoặc Safari hỗ trợ HTML5 Canvas.

A.2. Hướng dẫn biên dịch mã nguồn

Mở cửa sổ dòng lệnh Terminal hoặc Command Prompt tại thư mục gốc của dự án và thực thi lệnh Maven:

Lệnh biên dịch dự án với Apache Maven

```
mvn clean test      # Biên dịch mã nguồn và chạy 38 ca kiểm thử
mvn clean package   # Đóng gói sản phẩm thành file JAR thực thi
```

A.3. Hướng dẫn khởi chạy Hệ sinh thái Web Simulator (Khuyến nghị)

Để khởi chạy toàn bộ cụm 10 nút P2P, máy chủ REST API cổng 8000 và tự động mở giao diện web trên trình duyệt, chỉ cần nhấp đúp chuột vào tệp run_web.bat hoặc gõ lệnh:

Lệnh khởi chạy Web Simulator

```
java -cp target/classes com.pow.simulator.web.WebLauncher
# Hoặc nhấp đúp trực tiếp vào tệp run_web.bat tại thư mục gốc
```

Sau khi khởi chạy thành công, mở trình duyệt và truy cập các địa chỉ sau:

- **1. Bỏ phiếu cử tri:** http://localhost:8000/vote – Cổng bỏ phiếu cử tri trực tuyến
- **2. Sơ đồ 10 Nodes:** http://localhost:8000/nodes – Sơ đồ 10 nút P2P Mesh trực quan hóa hạt photon
- **3. Sổ Cái Explorer:** http://localhost:8000/ – Sổ cái chuỗi khối Explorer kiểm toán

A.4. Hướng dẫn triển khai mạng phân tán trên nhiều máy tính khác nhau

Để kết nối 3 máy tính của các bạn sinh viên (Máy 1 - Hoàng Nhật, Máy 2 - Trung Lương, Máy 3 - Quang Minh) trong phòng lab hoặc qua Internet:

- **Bước 1 (Seed Node):** Máy 1 (Hoàng Nhật) chạy run_seed_node.bat (cổng 8001), lấy địa chỉ IP LAN hoặc Tailscale.
- **Bước 2 (Peer Nodes):** Máy 2 (Trung Lương) & Máy 3 (Quang Minh): Điền IP:8001 vào peer_seed.txt rồi chạy run_peer_node.bat.
- **Bước 3 (Đồng bộ):** Hệ thống tự động bắt tay TCP, đồng bộ chuỗi khối dài nhất và hòa mạng P2P hoàn tất!